

MODULE 24

REST Endpoints and Request Mapping

Turn a REST design into working Spring MVC controllers, from request to response.





MODULE 24 OVERVIEW

Implement Spring MVC REST controllers for the Employee Management API designed in Module 13.

Scenario: build the @RestController layer that serves the Angular self-service portal's employee data.

DURATION & LAB



3 Hours Theory

Spring MVC internals, mapping annotations, data binding, responses, and testing.



1 Hour Guided Practice

Build and test EmployeeController's endpoints together, end to end.



Scenario

The Employee Management API's controller layer, called directly by the Angular portal.

MODULE ARC



Request Pipeline

Understand how DispatcherServlet and handler mapping route a request.



Bind and Respond

Read path variables, query parameters, and bodies; shape correct responses.



Verify It Works

Test controllers with MockMvc, Postman, and cURL.

KEY TAKEAWAY A REST controller layer is Module 13's design made real -- thin, correctly mapped, and verified by tests.

WHY REST CONTROLLERS MATTER

The controller is where an HTTP request from Angular first becomes Java code.



The Front Door

Every request from the Angular client enters the Employee Management API through a controller method.



Thin by Design

Controllers translate HTTP to Java calls and back -- business logic belongs in the service layer, not here.



Contract Made Real

Module 13's REST design (URIs, methods, status codes) becomes working code here for the first time.



Directly Testable

A well-built controller can be tested with MockMvc without starting a browser or an Angular app.

KEY TAKEAWAY A controller's only job is translating HTTP into a method call and a result back into HTTP -- nothing more.

WHY REST CONTROLLERS MATTER

REST Controllers are the entry point of your application. They receive HTTP requests, validate and map inputs, orchestrate business logic via services, and return well-structured responses. They translate HTTP into meaningful operations.



Single Entry Point for Clients

Provides a clear and consistent contract for web, mobile, and third-party clients.



Encapsulates HTTP Complexity

Handles HTTP methods, status codes, headers, and content negotiation for you.



Orchestrates Business Flow

Coordinates requests with services, repositories, and other system components.



Enforces Security & Validation

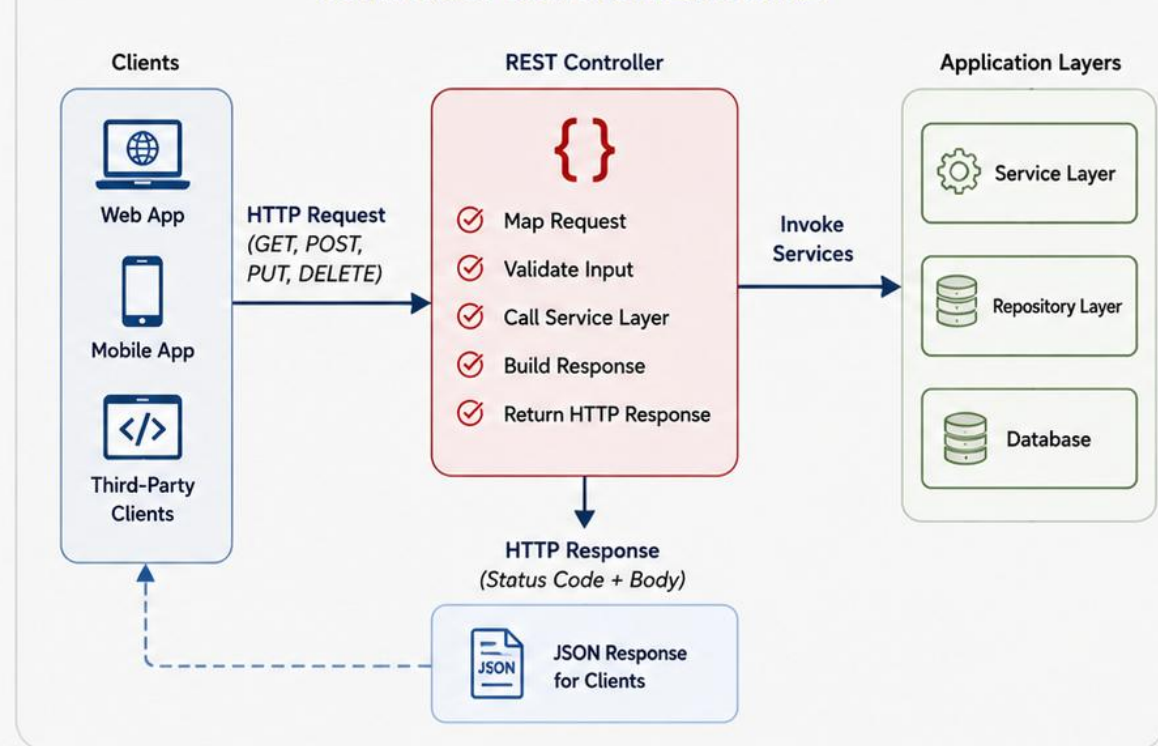
Applies authentication/authorization, input validation, and protects your domain logic.



Improves Maintainability & Scalability

Clean separation of concerns makes your APIs easier to evolve, test, and scale.

HOW A REST CONTROLLER WORKS



Key Takeaway: REST Controllers are the bridge between clients and your business logic. Well-designed controllers lead to secure, consistent, and developer-friendly APIs.

MODULE LEARNING OBJECTIVES

By the end of this module, you will build working Spring MVC REST controllers.



Build REST Controllers

Use `@RestController` and mapping annotations to implement the endpoints designed in Module 13.



Bind Request Data

Extract path variables, query parameters, and JSON bodies correctly and safely.



Shape Responses

Return correct status codes and headers using `ResponseEntity`.



Test Controllers

Verify controller behavior with `MockMvc` and manual tools like Postman and cURL.

KEY TAKEAWAY Every objective points at one deliverable: working `@RestController` code for the Employee Management API.

LEARNING OBJECTIVES

By the end of this module, you will be able to design RESTful APIs in Java that are well-structured, standards-compliant, and production-ready.



Understand REST Principles

Explain the REST architectural style and its constraints and how they guide API design.



Design Resource-Based APIs

Identify resources and design clean, intuitive, and consistent URIs that follow REST best practices.



Apply HTTP Standards

Use appropriate HTTP methods, status codes, headers, and content types to build robust APIs.



Structure Requests and Responses

Design effective JSON request and response payloads, including error handling and validation messages.



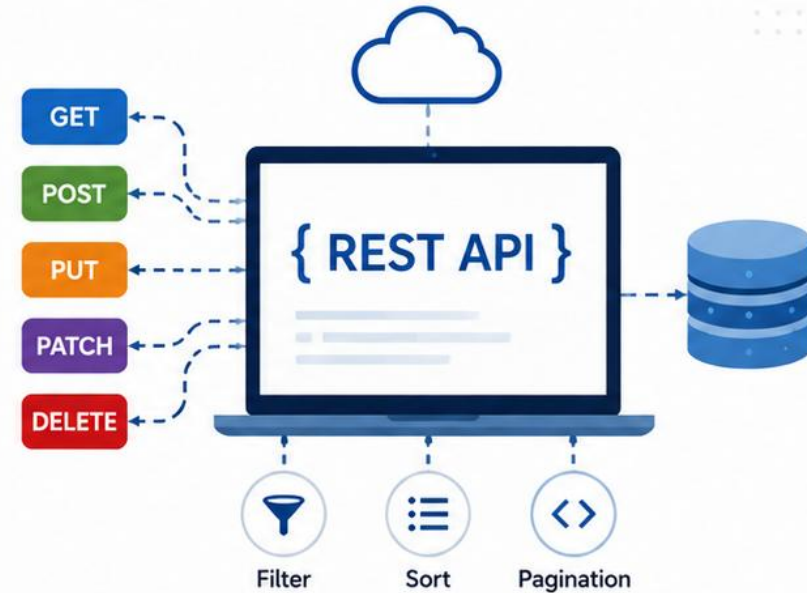
Implement Advanced API Features

Add pagination, filtering, sorting, and versioning strategies to support scalable and evolvable APIs.



Follow Best Practices

Apply industry best practices and avoid common design mistakes to build secure, maintainable, enterprise-grade APIs.



By the end of this module, you will be able to:

- ✓ Design RESTful APIs that are consistent and easy to use.
- ✓ Follow HTTP standards for communication.
- ✓ Build scalable, secure, and enterprise-ready APIs.
- ✓ Document APIs using OpenAPI specifications.
- ✓ Apply best practices to deliver high-quality API solutions.

SPRING MVC ARCHITECTURE

One request flows through a small, fixed set of collaborating components.



DispatcherServlet

The single entry point for every HTTP request into the Spring application.



Handler Mapping

Matches the incoming URI and method to the correct @RestController method.



Controller

Executes application logic by delegating to the service layer, then returns a result.



Message Converter

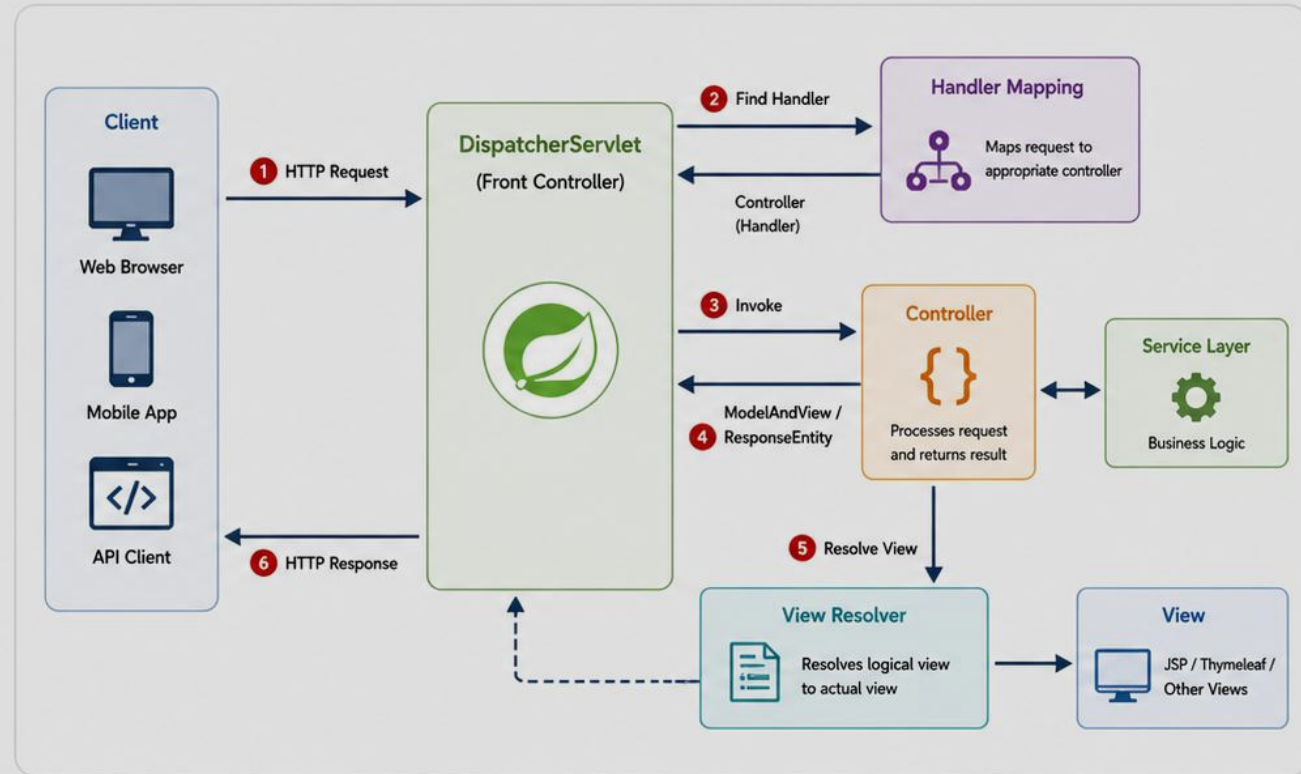
Serializes the returned Java object into JSON for the HTTP response body.

KEY TAKEAWAY Spring MVC's components each own one job -- knowing which one to touch is half of debugging a request.

SPRING MVC ARCHITECTURE

Spring MVC follows the Front Controller pattern to handle web requests. It provides a clean separation of concerns and a flexible, extensible design.

-  **1 Request**
The client sends an HTTP request to the application.
-  **2 DispatcherServlet (Front Controller)**
All requests are received by the DispatcherServlet, which acts as the front controller.
-  **3 Handler Mapping**
DispatcherServlet consults the HandlerMapping to find the appropriate Controller for the request.
-  **4 Controller (Handler)**
The Controller processes the request, invokes the service layer, and returns a ModelAndView or ResponseEntity.
-  **5 View Resolution**
If a view is returned, the ViewResolver maps the logical view name to the actual view (e.g., JSP, Thymeleaf) and renders the response.
-  **6 Response**
The DispatcherServlet sends the final HTTP response back to the client.



Key Takeaway: Spring MVC's Front Controller (DispatcherServlet) centralizes request handling, enabling a modular, maintainable, and testable web application architecture.

REST REQUEST PROCESSING OVERVIEW

From an Angular HTTP call to a JSON response, in six predictable stages.

1

Request Arrives

The Angular client sends an HTTP request to the API's base URL.

2

Servlet Dispatches

DispatcherServlet routes the request to the matching controller method.

3

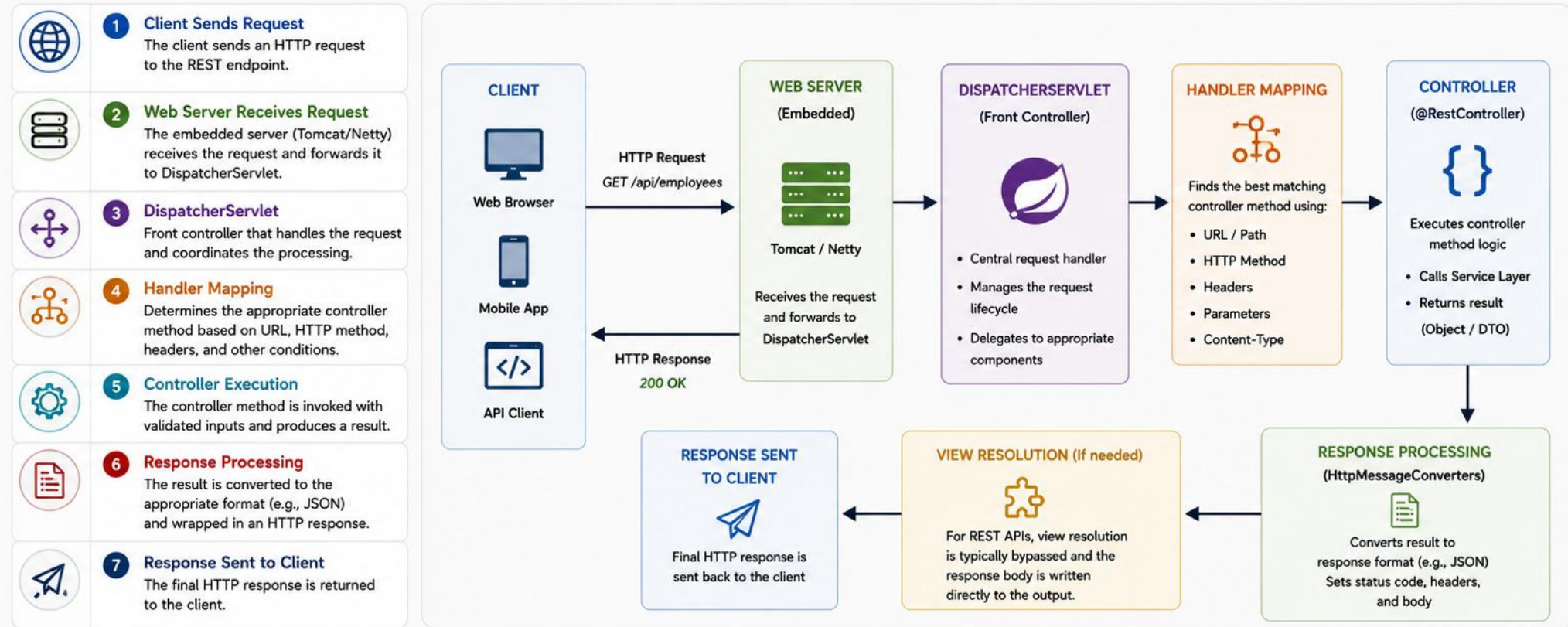
Data Binds

Path variables, query parameters, and the request body populate method parameters.

KEY TAKEAWAY Every request follows the same six stages -- memorize the order, and every bug becomes easier to localize.

REST REQUEST PROCESSING OVERVIEW

A REST request goes through a well-defined pipeline in a Spring Boot application. Each component has a specific responsibility to handle the request and produce the response.



Key Takeaway: Spring MVC processes REST requests through a structured pipeline. Each component plays a critical role in ensuring that requests are handled efficiently and responses are returned consistently.



Benefits

- ✓ Separation of concerns
- ✓ Testability
- ✓ Extensibility
- ✓ Consistent request handling

DISPATCHERSERVLET

Spring Boot's single front-controller servlet that every incoming HTTP request passes through.



One Front Door

A single servlet instance receives every HTTP request the application handles, regardless of endpoint.



Auto-Configured

Spring Boot registers and configures DispatcherServlet automatically -- no XML or manual servlet setup needed.



Delegates, Doesn't Decide

It routes to handler mapping and controllers rather than containing any business logic itself.

KEY TAKEAWAY DispatcherServlet is auto-configured by Spring Boot -- students rarely configure it directly, but should know it's there.

DISPATCHERSERVLET

DispatcherServlet is the front controller in Spring MVC. It is a central servlet that receives all incoming HTTP requests, delegates them to the appropriate components, and returns the response to the client.

KEY RESPONSIBILITIES



Front Controller

Acts as a single entry point for all requests in the application.



Request Handling

Coordinates the request processing workflow with other components.



Component Delegation

Delegates to HandlerMapping, Controllers, ViewResolvers, and other components.



Response Management

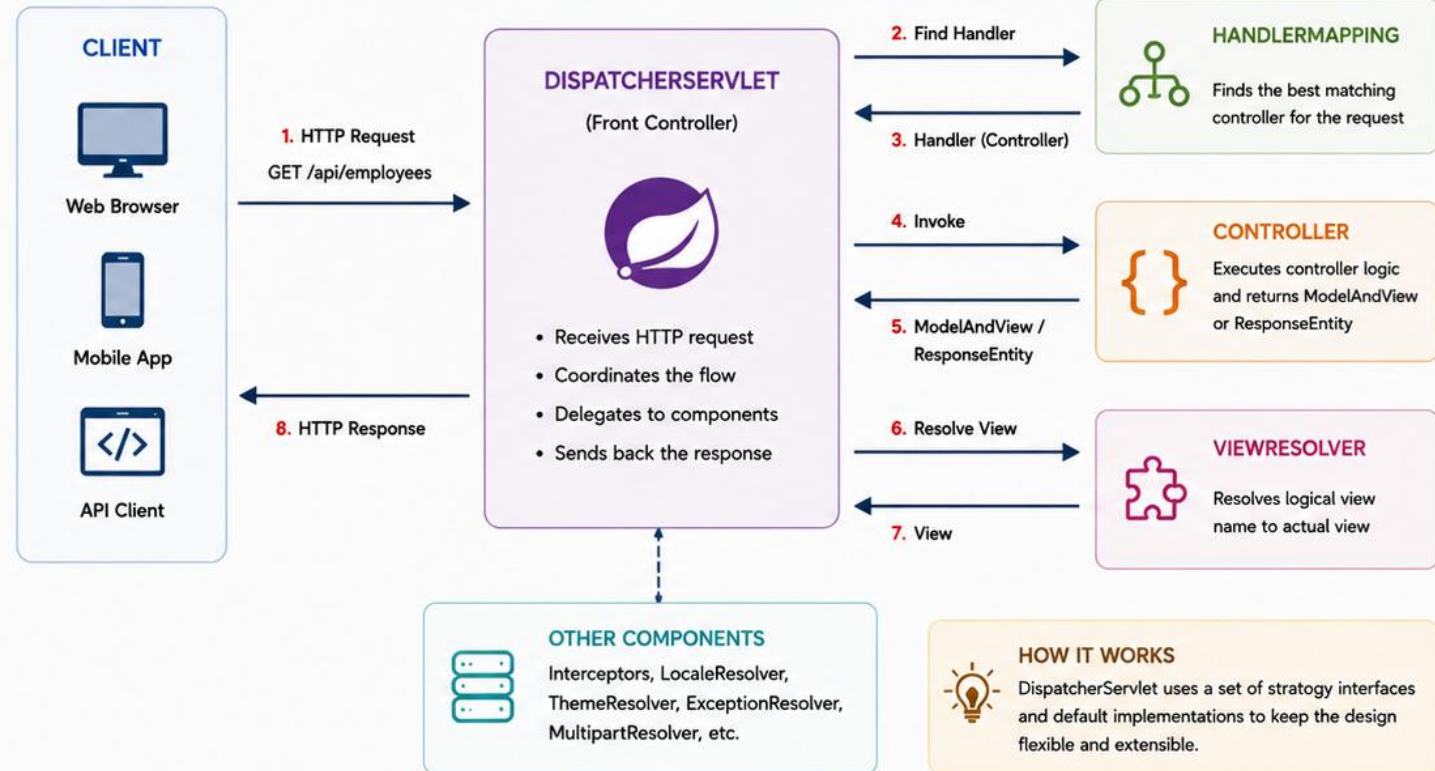
Handles the view resolution or response serialization and sends the final response.



Extensible & Configurable

Highly configurable and allows customization via strategies and interceptors.

REQUEST PROCESSING FLOW



KEY TAKEAWAY: DispatcherServlet is the heart of Spring MVC. It simplifies the complexity of request handling by centralizing the workflow and delegating tasks to specialized components.



Centralized request handling



Flexible and extensible



Improves maintainability and testability

CONTROLLER LAYER RESPONSIBILITIES

A controller's contract with the rest of the application is narrow and deliberate.



Accept the Request

Bind path variables, query parameters, and the request body to typed method parameters.



Delegate to Service

Call the appropriate EmployeeService method, passing the bound data along.



Shape the Response

Wrap the service's result in a ResponseEntity with the correct status code and headers.

KEY TAKEAWAY If a controller method is more than a few lines of delegation, business logic has leaked into the wrong layer.

Controller Layer Responsibilities

The Controller layer is the **entry point** for all HTTP requests in a Spring Boot application.

Key Responsibilities



Handle HTTP Requests

Map incoming HTTP requests to appropriate handler methods using @RequestMapping and related annotations.



Extract & Validate Input

Extract path variables, query parameters, headers, and request bodies. Validate input and ensure data integrity.



Delegate to Service Layer

Delegate business logic to the Service layer. Controllers should not contain business rules.



Build & Return HTTP Responses

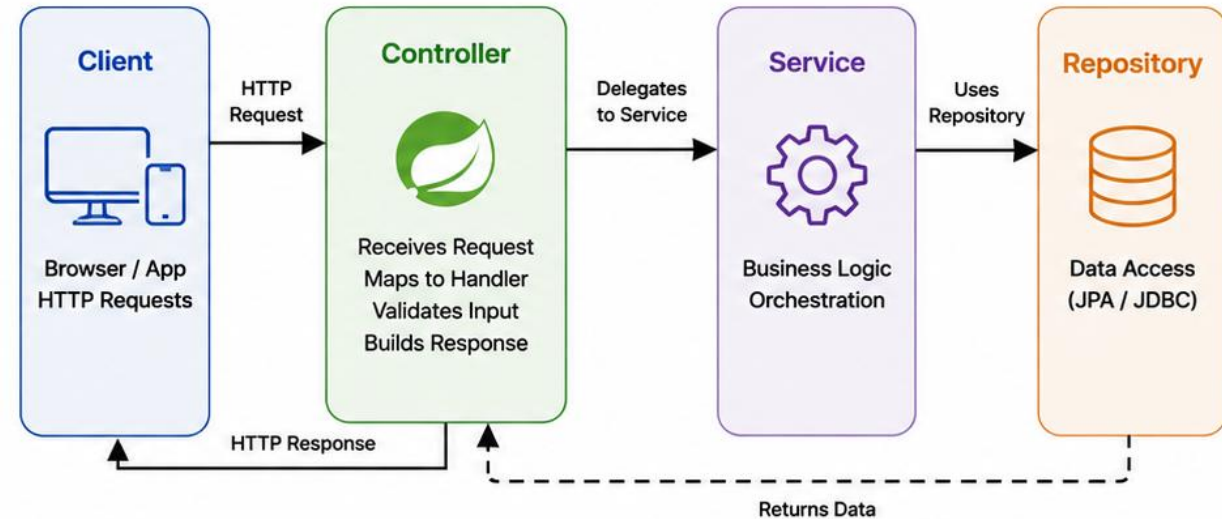
Return appropriate HTTP status codes, headers, and response bodies (typically JSON).



Handle Exceptions

Handle or delegate exceptions and return meaningful error responses to the client.

Where the Controller Fits



Remember

Controllers are thin: they should handle HTTP concerns and input/output. All business logic belongs in the **Service layer**.



Controllers connect the outside world to your application. | Keep them focused, simple, and consistent.

@RestController

One annotation that marks a class as a controller and serializes every return value to JSON automatically.



Marks the Class

Tells Spring this class contains request-handling methods, and registers it as a bean.



Bakes In @ResponseBody

Every method's return value is serialized directly into the response body -- no view resolution.



Built for APIs

Unlike plain @Controller, which is built for rendering HTML views, @RestController is built for JSON APIs.

KEY TAKEAWAY @RestController is @Controller plus @ResponseBody on every method -- that combination is what makes it JSON by default.

@RestController

The @RestController annotation is a specialized version of @Controller that combines @Controller and @ResponseBody.

KEY POINTS

-  Marks a class as a **RESTful web service controller**.
-  All methods return data directly to the **HTTP response body** (typically JSON or XML).
-  Eliminates the need to annotate each method with @ResponseBody.
-  Ideal for building REST APIs in Spring Boot.



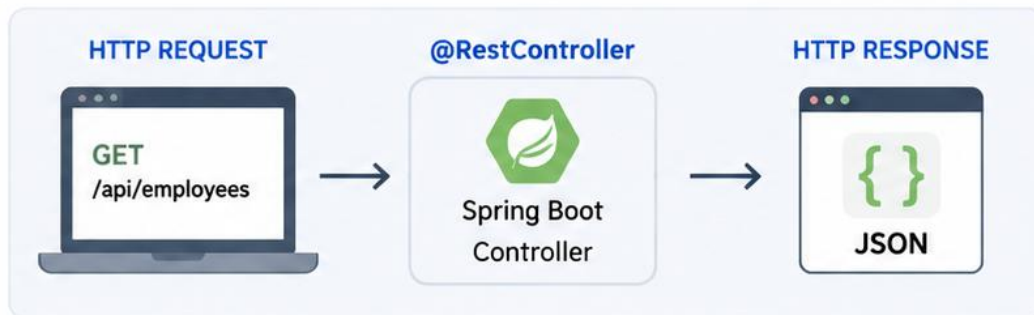
BEST PRACTICE

Use @RestController for REST APIs to ensure consistent response behavior and cleaner code.

```
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/employees")           // Returns JSON
                                           // by default

@GetMapping
public List<Employee> getAllEmployees() {
    return employeeService.getAllEmployees();
}
```



REMEMBER | @RestController = @Controller + @ResponseBody



@REQUESTMAPPING

The general-purpose annotation for binding a URI (and optionally a method) to a controller or method.



Class-Level Base Path

@RequestMapping("/api/v1/employees") on the class prefixes every method's mapping.



Method + Path Together

@RequestMapping(method = RequestMethod.GET, value = "/{id}") works, but is verbose.



Superseded by Shortcuts

@GetMapping, @PostMapping, and friends are @RequestMapping specializations with cleaner syntax.

KEY TAKEAWAY @RequestMapping still works everywhere, but the method-specific annotations read cleaner for almost every real case.

@RequestMapping

@RequestMapping is a versatile annotation used to map HTTP requests to handler methods or controller classes.

KEY POINTS

-  Can be applied at **class level** to define a base URL path for all methods.
-  Can be applied at **method level** to map specific HTTP methods and paths.
-  Supports **path patterns**, query parameters, headers, and content types.
-  Provides a **flexible** way to define REST endpoints in Spring MVC.

Example

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @GetMapping
    public List<EmployeeDto> getAllEmployees() {
        return employeeService.getAllEmployees();
    }

    @PostMapping
    public ResponseEntity<EmployeeDto> createEmployee(
        @RequestBody EmployeeCreateDto dto) {
        EmployeeDto created = employeeService.create(dto);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }
}
```

MAPPING FLOW



COMMON USE CASES

- Define base paths for controllers
- Map multiple HTTP methods
- Handle different content types
- Version API endpoints

@GETMAPPING THROUGH @DELETEMAPPING

Five method-specific shortcuts, one per HTTP verb, each a specialization of @RequestMapping.

Annotation	HTTP Method	Example
@GetMapping	GET	@GetMapping("/{id}")
@PostMapping	POST	@PostMapping
@PutMapping	PUT	@PutMapping("/{id}")
@PatchMapping	PATCH	@PatchMapping("/{id}")
@DeleteMapping	DELETE	@DeleteMapping("/{id}")

KEY TAKEAWAY Pick the annotation that matches the HTTP method your Module 13 design already specified -- there's no ambiguity left to resolve.



@GetMapping

@GetMapping is a composed annotation that maps HTTP GET requests to a specific handler method.

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @GetMapping
    public List<EmployeeDTO> getAllEmployees() {
        return employeeService.getAllEmployees();
    }
}
```

 GET /api/employees



Use **@GetMapping** to read data.
GET requests are **safe** and **should not** modify server state.



Purpose

Handles HTTP **GET** requests to retrieve resources from the server.



Key Characteristics

- Used for reading data
- Safe (no side effects)
- Idempotent
- Can be cached



Best Practices

- Use meaningful URIs
- Return appropriate response codes
- Support pagination, filtering, and sorting
- Avoid heavy computation in GET calls



@PostMapping

HANDLE HTTP POST REQUESTS TO CREATE RESOURCES

@PostMapping maps HTTP **POST** requests to controller methods and is used to create new resources on the server.



Creates New Resources

Used to create and persist a new resource.



Accepts Request Body

Typically accepts JSON payload using **@RequestBody**.



Not Idempotent

Each POST request creates a new resource.



Common Response

Returns **201 Created** with Location header.

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @PostMapping()
    public ResponseEntity<EmployeeDto> createEmployee(
        @RequestBody @Valid EmployeeDto employeeDto) {
        EmployeeDto created = employeeService.create(employeeDto);
        URI location = ServletUriComponentsBuilder
            .fromCurrentRequest()
            .path("/{id}")
            .buildAndExpand(created.getId())
            .toUri();
        return ResponseEntity.created(location).body(created);
    }
}
```



Key Takeaway: **@PostMapping** handles HTTP POST requests to create new resources. It accepts a request body, processes it, and returns **201 Created** with the newly created resource.



@PutMapping

Handle Full Updates to an Existing Resource



Purpose

@PutMapping handles HTTP PUT requests used to replace an existing resource completely.

```
1 @PutMapping("/employees/{id}")
2 public ResponseEntity<EmployeeDto> updateEmployee(
3     @PathVariable Long id,
4     @RequestBody @Valid EmployeeDto employeeDto) {
5
6     Employee updated = employeeService.updateEmployee(id, employeeDto);
7     return ResponseEntity
8         .ok(updated);
9 }
10
11
12
```



- PUT replaces the entire resource.
- The resource must exist; otherwise return **404 Not Found**.
- The client should send the full updated representation.

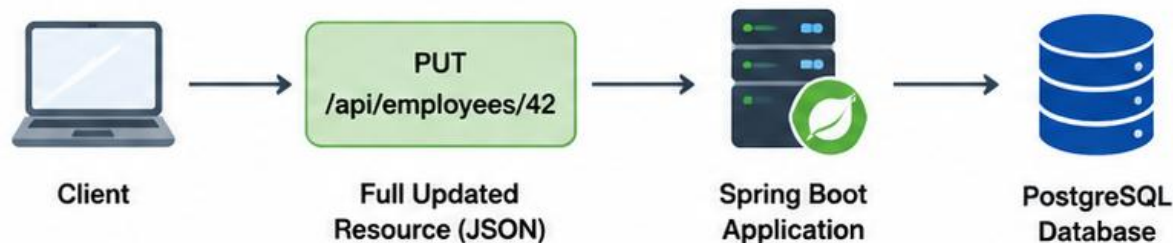


Key Takeaway

Use **@PutMapping** to fully replace an existing resource. It is idempotent and ensures the resource state matches the provided representation.



PUT Request Flow



PUT vs Other Methods

Method	Purpose	Idempotent	Creates Resource
GET	Retrieve a resource	Yes	No
POST	Create a new resource	No	Yes
PUT	Replace an existing resource	Yes	No
PATCH	Partially update a resource	Yes	No
DELETE	Delete a resource	Yes	No

@PatchMapping

MODULE 24

REST Endpoints and
Request Mapping

@PatchMapping handles HTTP **PATCH** requests to apply partial updates to an existing resource.



KEY POINTS

- Used to update specific fields of an existing resource.
- Less risk than PUT since it does not require the full resource.
- Request body contains only the fields to be modified.
- Idempotent: multiple identical PATCH requests produce the same result.
- Use for partial updates in real-world applications (e.g., profile updates, settings).

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @PatchMapping("/{id}")
    public ResponseEntity<EmployeeDto> updateEmployee(
        @PathVariable Long id,
        @RequestBody EmployeeUpdateDto updateDto) {

        // Apply partial update and return updated resource
        EmployeeDto updated = employeeService.patchUpdate(id, updateDto);
        return ResponseEntity.ok(updated);
    }
}
```



HTTP PATCH EXAMPLE

PATCH /api/employees/101

Content-Type: application/json

```
{
  "phone": "987-654-3210",
  "department": "Engineering"
}
```



RESPONSE (200 OK)

```
{
  "id": 101,
  "name": "John Doe",
  "email": "john.doe@example.com",
  "phone": "987-654-3210",
  "department": "Engineering"
}
```



When to Use PATCH

Use @PatchMapping when only a subset of a resource needs to be updated—ideal for partial updates like profile edits, status changes, or preferences.

Best Practices

- ✓ Validate only the fields being updated.
- ✓ Document PATCH behavior clearly in your API.
- ✓ Avoid over-fetching—update only what is necessary.
- ✓ Ensure the operation is idempotent.



@DeleteMapping

Maps HTTP DELETE requests to handler methods that remove resources.



Purpose

Used to delete a specific resource identified by the URI.



Key Points

- Idempotent: Multiple DELETE calls for the same resource should have the same effect (resource is gone).
- Typically returns 204 No Content when deletion is successful.
- Use meaningful status codes for error scenarios (404, 409, etc.).



Best Practice

Delete operations should be idempotent and target a specific resource URI.

Example

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteEmployee(
        @PathVariable Long id) {
        employeeService.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}
```

HTTP Request

```
DELETE /api/employees/123
Host: api.example.com
Authorization: Bearer <token>
```

HTTP Response (Success)

```
HTTP/1.1 204 No Content
Content-Length: 0
```

HTTP Response (Not Found)

```
HTTP/1.1 404 Not Found
Content-Type: application/json
{
  "timestamp": "2025-05-19T10:15:30Z",
  "status": 404,
  "error": "Not Found",
  "message": "Employee not found",
  "path": "/api/employees/123"
}
```



REQUEST MAPPING WORKFLOW

How Spring picks exactly one method to handle an incoming request, even with many candidates.



Path Matching First

Spring finds every mapping whose path pattern matches the incoming URI, including path variables.



Method Narrows It

Among path matches, the HTTP method (GET, POST, ...) narrows to exactly one method.



405 on Method Mismatch

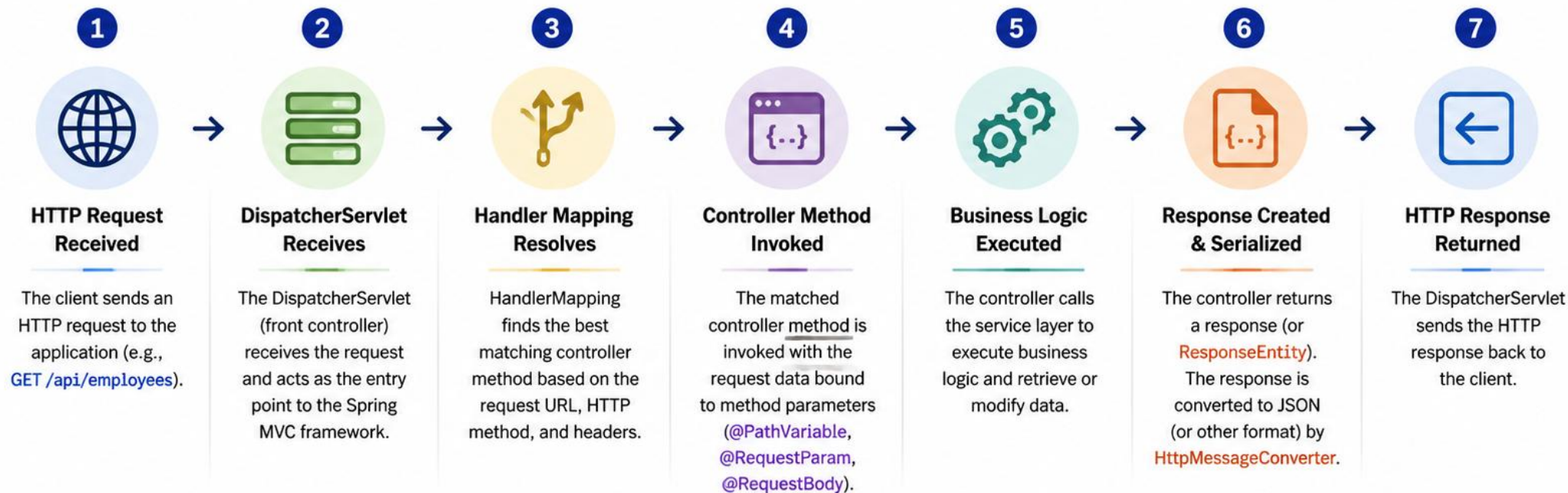
A matching path with no matching method returns 405 Method Not Allowed, not 404.

KEY TAKEAWAY Spring matches on path AND method together -- two endpoints can share a path as long as their HTTP methods differ.

Request Mapping Workflow



How an HTTP request is received, mapped, processed, and a response is returned in Spring MVC.



Example Mapping

```
GET /api/employees/123 → @GetMapping("/api/employees/{id}")
public Employee getEmployee(@PathVariable Long id) { ... }
```



Key Takeaway: Spring MVC handles request routing, parameter binding, and response serialization — allowing developers to focus on business logic in controller methods.

TRY IT YOURSELF — EXERCISE 1: PLAN THE CONTROLLER MAPPINGS

Reinforces: the Spring MVC request path, @RestController, and the mapping annotations you just walked, planned for CustomerController.

STEPS (notes/lab24-mapping-plan.md)

Step	What To Do
1 — Class Annotations	Write the class-level annotations, including the base @RequestMapping path.
2 — Method Table	List each handler method with its mapping annotation and URI suffix.
3 — Request Journey	Name the stages a request passes through from DispatcherServlet to your method.
4 — Scope	Plan only — no controller class created in this pre-lab.

Controller skeleton

```
@RestController @RequestMapping("/api/customers")
@GetMapping("/{id}")  getOne      @GetMapping  list
@PostMapping create    @PutMapping("/{id}") update
```

TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-controller-mapping-plan.md (workspace: examples/module-24-exercises).

PATH VARIABLES AND @PATHVARIABLE

A URI template placeholder, bound directly into a typed Java method parameter.



URI Template Placeholder

{employeeId} in
@GetMapping("/{employeeId}")
marks a segment of the path as
variable.



Bound by @PathVariable

public EmployeeResponse
getEmployee(@PathVariable
Long employeeId) binds that
segment into a typed
parameter.



Automatic Type Conversion

Spring converts the string '42'
from the URI into a Long
automatically -- a non-numeric
value returns 400.

KEY TAKEAWAY @PathVariable's parameter name only needs to match the {placeholder} name automatically when they're spelled identically.

PATH VARIABLES

Path variables are placeholders in the URI that capture values to identify specific resources.

Key Points

- Used to identify a specific resource in a collection.
- Defined in the URI template using `{variableName}`.
- Captured and passed to the controller method.
- Provides clean, flexible, and RESTful URLs.

Examples of Path Variables in URIs

✓ GET /api/employees/ 101	→ Get employee with ID 101
✓ GET /api/employees/ 101 /documents/ 55	→ Get document 55 of employee 101
✓ GET /api/departments/ 10 /employees	→ Get all employees in department 10
✓ DELETE /api/employees/ 101	→ Delete employee with ID 101



Best Practices

- Use nouns in URIs, not verbs.
- Use meaningful and consistent variable names.
- Use path variables for resource identification, not optional filters.

How It Works



Controller Example

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @GetMapping("/{id}")
    public ResponseEntity<EmployeeDto> getEmployee(
        @PathVariable Long id) {
        EmployeeDto employee = employeeService.findById(id);
        return ResponseEntity.ok(employee);
    }
}
```

Captures the value from the URI and passes it to the method parameter.

Request and Result

HTTP Request

```
GET /api/employees/101
Accept: application/json
```



HTTP Response (200 OK)

```
{
  "id": 101,
  "name": "John Doe",
  "email": "john.doe@example.com"
}
```

QUERY PARAMETERS AND @REQUESTPARAM

Optional, named request modifiers -- exactly how Module 13's pagination, filtering, and sorting arrive.



Optional by Nature

?department=ENG&page=0&size=20 -- query parameters modify a request without changing its resource.



Bound by @RequestParam

```
public  
List<EmployeeResponse>  
list(@RequestParam(required =  
false) String department) binds  
it directly.
```



Defaults Avoid Nulls

@RequestParam(defaultValue = "0") int page avoids null-checking every optional parameter manually.

KEY TAKEAWAY Give @RequestParam a sensible default instead of making it required whenever the parameter is genuinely optional.

QUERY PARAMETERS

Query parameters are optional key-value pairs appended to the URI to filter, sort, paginate, or customize the request.



Key Points

- Added to the URI after a “?” and separated by “&”.
- Used for non-hierarchical data such as filters, sorting, pagination.
- Mapped in controller methods using `@RequestParam`.
- Optional by default; can be required when needed.
- Improves API flexibility without changing the endpoint structure.

Examples of Query Parameters

GET	/api/employees?department=IT	→ All employees in IT department
GET	/api/employees?status=ACTIVE&role=DEV	→ Active developers
GET	/api/employees?sort=name,asc	→ Sorted by name (ascending)
GET	/api/employees?page=0&size=10	→ Pagination: page 0, 10 items
GET	/api/employees?search=john	→ Search employees by name



Best Practices

- ✓ Use meaningful and consistent parameter names.
- ✓ Use default values for optional parameters.
- ✓ Validate and sanitize all input parameters.
- ✓ Avoid too many parameters—use filters or request body for complex queries.

How It Works



Controller Example

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {
    @GetMapping
    public ResponseEntity<List<EmployeeDto>> getEmployees(
        @RequestParam(required = false) String department,
        @RequestParam(required = false) String status,
        @RequestParam(required = false, defaultValue = "0") int page,
        @RequestParam(required = false, defaultValue = "10") int size,
        @RequestParam(required = false, defaultValue = "name,asc") String sort
    ) {
        // Use parameters to filter, sort, and paginate results
        List<EmployeeDto> employees = employeeService.getEmployees(
            department, status, page, size, sort);
        return ResponseEntity.ok(employees);
    }
}
```

HTTP Request

```
GET /api/employees?department=IT&status=
ACTIVE&page=0&size=10&sort=name,asc
Accept: application/json
```



HTTP Response (200 OK)

```
{
  "content": [ ... ],
  "page": 0,
  "size": 10,
  "totalElements": 125,
  "totalPages": 13
}
```

REQUEST BODIES AND @REQUESTBODY

The JSON payload of a POST, PUT, or PATCH, deserialized directly into a typed Java object.



The JSON Payload

A POST or PUT/PATCH carries the new or updated employee data in its HTTP body.



Bound by @RequestBody

create(@RequestBody
EmployeeCreateRequest
request) deserializes the JSON
body automatically.



400 on Malformed JSON

Invalid or unparseable JSON in
the body returns 400 before the
controller method ever runs.

KEY TAKEAWAY @RequestBody triggers Jackson deserialization -- the JSON's field names must line up with the target class's fields.

REQUEST BODIES

Request bodies carry the data sent from the client to the server, typically in POST, PUT, or PATCH requests.

Key Points

- Used to send structured data in the body of an HTTP request.
- Commonly in JSON format for REST APIs.
- Not used with GET or DELETE (no body by convention).
- Mapped to method parameters using **@RequestBody**.
- Enables creation, updates, and complex operations.

HTTP Methods and Request Bodies		
HTTP Method	Body Used?	Purpose
GET	No	Retrieve resources
POST	Yes	Create a new resource
PUT	Yes	Replace an existing resource
PATCH	Yes	Partial update of a resource
DELETE	No (by convention)	Remove a resource

Best Practices

- ✓ Use JSON as the preferred format.
- ✓ Validate all input data.
- ✓ Keep request bodies simple and consistent.
- ✓ Return meaningful validation errors.
- ✓ Keep business logic out of the controller.

How It Works



Controller Example

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @PostMapping
    public ResponseEntity<EmployeeDto> createEmployee(
        @RequestBody EmployeeDto employeeDto) {
        // employeeDto is automatically deserialized from the request body
        EmployeeDto saved = employeeService.create(employeeDto);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }
}
```

@RequestBody binds the request body to the method parameter.

Example: Create Employee

HTTP Request

```
POST /api/employees HTTP/1.1
Host: api.example.com
Content-Type: application/json
Accept: application/json
```

```
{
  "name": "John Doe",
  "email": "john.doe@example.com",
  "departmentId": 10,
  "designation": "Developer"
}
```

HTTP Response (201 Created)

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/employees/101
```

```
{
  "id": 101,
  "name": "John Doe",
  "email": "john.doe@example.com",
  "departmentId": 10,
  "designation": "Developer"
}
```

JSON DESERIALIZATION AND JACKSON OBJECT MAPPING

Jackson is the library doing the actual work behind every @RequestBody and @RestController return value.



Included by Default

spring-boot-starter-web pulls in Jackson automatically -- no extra dependency or configuration needed.



Two-Way Mapping

The same Jackson ObjectMapper deserializes request JSON and serializes response objects to JSON.



Matches by Field Name

JSON keys map to Java fields (or getter/setter pairs) with matching names -- camelCase on both sides.

KEY TAKEAWAY Jackson maps JSON to Java by field name and type -- getters/setters or public fields, matched automatically, no manual parsing.

Jackson Object Mapping



Jackson is the default JSON library in Spring Boot.
It converts Java objects to JSON (**serialization**) and
JSON to Java objects (**deserialization**).

Key Responsibilities

- Convert Java objects to JSON (Serialization)
- Convert JSON to Java objects (Deserialization)
- Handle complex types, collections, null values, and date/time formats
- Used automatically by Spring MVC in @RequestBody and responses

Java Object

```
public class Employee {  
    private Long id;  
    private String name;  
    private String email;  
    private LocalDate hireDate;  
    private BigDecimal salary;  
}
```

Serialization
(Object → JSON)



Deserialization
(JSON → Object)

JSON

```
{  
  "id": 101,  
  "name": "Alice Johnson",  
  "email": "alice@example.com",  
  "hireDate": "2024-01-15",  
  "salary": 85000.00  
}
```

Serialization (Object to JSON)

```
ObjectMapper mapper = new ObjectMapper();  
Employee emp = new Employee(101L, "Alice Johnson",  
    "alice@example.com", LocalDate.of(2024, 1, 15),  
    new BigDecimal("85000.00"));  
String json = mapper.writeValueAsString(emp);  
// Java Object → JSON
```

Deserialization (JSON to Object)

```
ObjectMapper mapper = new ObjectMapper();  
String json = "{\"id\":101,\"name\":\"Alice Johnson\",  
    \"email\":\"alice@example.com\", \"hireDate\":\"2024-01-15\",  
    \"salary\":85000.00}";  
Employee emp = mapper.readValue(json, Employee.class);  
// JSON → Java Object
```

Key Annotations

- @JsonProperty("name") Custom JSON property name
- @JsonIgnore Ignore field in JSON
- @JsonFormat Control date/time and number formats
- @JsonInclude Include non-null or non-empty fields
- @JsonNaming Apply naming strategies (SNAKE_CASE, etc.)



Key Takeaway: Jackson simplifies JSON handling in Java applications by automatically mapping between Java objects and JSON.

TRY IT YOURSELF — EXERCISE 2: BIND PATH, QUERY AND BODY

Reinforces: @PathVariable, @RequestParam, @RequestBody, and Jackson deserialization as you just saw them, applied to the CRM endpoints.

STEPS (notes/lab24-binding-table.md)

Step	What To Do
1 — Binding Table	For each handler, name the annotation that binds each input.
2 — Optional Filter	Give the list filter a default so a missing parameter is not an error.
3 — Jackson Mapping	Say what Jackson needs to turn the request JSON into a CustomerDto.
4 — Wrong Type	Say what happens when a query parameter cannot be converted.

Binding annotations

```
getOne(@PathVariable String id)           // from the URI
list(@RequestParam(defaultValue="") String status) // from the query
create(@RequestBody CustomerDto dto)       // from the JSON body
```

TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-binding-table.md (workspace: examples/module-24-exercises).

RESPONSE SERIALIZATION

Whatever a controller method returns gets turned into the JSON body of the HTTP response.



Return Value Becomes Body

@RestController's implicit @ResponseBody means the returned object is serialized directly.



Never Return Entities

Returning a JPA entity directly risks leaking internal fields and lazy-loading exceptions into the JSON.



Return the DTO

Returning EmployeeResponse keeps the wire format decoupled from the database schema.

KEY TAKEAWAY Return the response DTO, not the JPA entity -- serialization happens on whatever type the method declares.



RESPONSE SERIALIZATION

Response serialization is the process of converting Java objects into JSON and sending them back to the client as the HTTP response body.



Key Points

- Spring Boot uses Jackson by default to serialize responses to JSON.
- Called automatically by Spring MVC when returning objects from @RestController.
- Respects annotations like @JsonProperty, @JsonFormat, and @JsonInclude.
- Supports complex types, collections, nested objects, and date/time formats.
- Content-Type header is set to application/json by default.

Example: Controller Method

```
@GetMapping("/employees/{id}")  
public ResponseEntity<Employee> getEmployee(@PathVariable Long id) {  
    Employee employee = employeeService.getEmployeeById(id);  
    return ResponseEntity.ok(employee);  
}
```

Serialized HTTP Response

```
HTTP/1.1 200 OK  
Content-Type: application/json  
Date: Tue, 20 May 2025 10:30:00 GMT  
  
{  
  "id": 101,  
  "name": "Alice Johnson",  
  "email": "alice@example.com",  
  "salary": 85000.00,  
  "hireDate": "2024-01-15"  
}
```

Where Serialization Happens



Remember: Any Java object returned from a @RestController is automatically serialized to JSON unless you customize it.

★ **Best Practice:** Use DTOs for responses to control the data exposed to clients and improve security and performance.

RESPONSEENTITY

A wrapper type that gives a controller method explicit control over status code, headers, and body together.



Status, Headers, Body

`ResponseEntity<EmployeeResponse>` bundles all three pieces of an HTTP response into one return value.



Building One

`ResponseEntity.status(HttpStatus.CREATED).header("Location", uri).body(response)` builds a 201 with a Location header.



Full Control When Needed

Use `ResponseEntity` whenever the status code isn't a plain 200, or a header must be set explicitly.

KEY TAKEAWAY Returning a plain object always sends 200 -- `ResponseEntity` is how a controller sends any other status code deliberately.

RESPONSEENTITY



ResponseEntity is a Spring class that represents the entire HTTP response, including status code, headers, and body.

Key Points

- ✓ Encapsulates status code, headers, and body.
- ⚙ Provides full control over the HTTP response.
- 📄 Preferred for REST APIs with dynamic responses.
- 🛡 Improves API clarity, flexibility, and consistency.

ResponseEntity Structure

ResponseEntity<T> = HTTP Status + Headers + Body



ResponseEntity gives you complete control over the HTTP response returned to the client.

Example Controller Method

```
@GetMapping("/employees/{id}")
public ResponseEntity<EmployeeDto> getEmployee(@PathVariable Long id) {
    return employeeService.findById(id)
        .map(employee ->
            ResponseEntity.ok() // 200 OK
                .body(employee))
        .orElse(ResponseEntity
            .status(HttpStatus.NOT_FOUND) // 404 Not Found
                .body(null));
}
```

Example JSON Response (200 OK)

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-cache
Date: Tue, 20 May 2025 10:30:00 GMT
{
  "id": 101,
  "name": "Alice Johnson",
  "email": "alice@example.com",
  "hireDate": "2024-01-15",
  "salary": 85000.00
}
```

Common Status Codes

Category	Code	Meaning	When to Use
2xx Success	200 OK	Request succeeded	Successful GET/PUT/PATCH/DELETE
2xx Success	201 Created	Resource created	After successful POST
4xx Client Error	400 Bad Request	Invalid request	Validation or malformed request
4xx Client Error	404 Not Found	Resource not found	When resource does not exist
5xx Server Error	500 Internal Server Error	Server error	Unexpected server-side failure

Common Builder Methods

ok(body)	200 OK with body
created(uri)	201 Created with Location header
status(code)	Custom status code
header(name, value)	Add a header
body(body)	Set response body
noContent().build()	204 No Content
badRequest().body(error)	400 Bad Request with body

★ **Best Practice:** Use ResponseEntity for all REST endpoints to maintain consistency and provide meaningful HTTP responses.

HTTP STATUS CODES IN SPRING

The `HttpStatus` enum gives every status code from Module 13 a type-safe, readable Java name.

Operation	HttpStatus Constant	Numeric Code
Successful GET	HttpStatus.OK	200
Successful POST (create)	HttpStatus.CREATED	201
Successful DELETE	HttpStatus.NO_CONTENT	204
Validation failure	HttpStatus.BAD_REQUEST	400
Resource not found	HttpStatus.NOT_FOUND	404

KEY TAKEAWAY Use the `HttpStatus` enum constant, never a bare integer -- it self-documents and the compiler catches typos.

HTTP STATUS CODES IN SPRING



Spring provides built-in support for returning standard HTTP status codes using `ResponseEntity`, `@ResponseStatus`, or `exceptions`.

Ways to Set HTTP Status Codes in Spring



1. Using `ResponseEntity`

Return full control over status, headers, and body.



2. Using `@ResponseStatus`

Annotate controller methods or custom exceptions.



3. Using `Exceptions`

Throw exceptions and map them to appropriate status codes.

Common Status Code Categories

Category	Range	Purpose	Examples
1xx Informational	100 – 199	Request received, continuing process	100 Continue
2xx Success	200 – 299	Request succeeded	200 OK 201 Created
3xx Redirection	300 – 399	Further action needed by client	301 Moved Permanently 304 Not Modified
4xx Client Error	400 – 499	Invalid request from client	400 Bad Request 404 Not Found
5xx Server Error	500 – 599	Server failed to fulfill request	500 Internal Server Error 503 Service Unavailable

Example: Using `ResponseEntity`

```
@GetMapping("/employees/{id}")
public ResponseEntity<Employee> getEmployee(@PathVariable Long id) {
    return employeeService.findById(id)
        .map(emp -> ResponseEntity
            .ok(emp)) // 200 OK
        .orElse(ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(null)); // 404 Not Found
}
```



Tip: Choose the most specific and meaningful status code. It improves API clarity and client experience.

Common Status Codes in Spring

✓ 200 OK	Request succeeded.	⚠ 400 Bad Request	Invalid request or malformed input.	✗ 500 Internal Server Error	Unexpected server error.
✓ 201 Created	Resource created successfully.	⚠ 401 Unauthorized	Authentication is required.	✗ 502 Bad Gateway	Invalid response from upstream server.
✓ 204 No Content	Request succeeded, no content to return.	⚠ 403 Forbidden	Access is denied.	✗ 503 Service Unavailable	Server is unavailable.
📄 301 Moved Permanently	Resource moved permanently.	⚠ 404 Not Found	Requested resource not found.	✗ 504 Gateway Timeout	Upstream server timeout.
📄 304 Not Modified	Resource not modified since last request.	⚠ 409 Conflict	Request conflicts with current state.		



Best Practice: Use appropriate HTTP status codes consistently across all endpoints. It is a core part of a well-designed REST API.

SETTING RESPONSE HEADERS

ResponseEntity's builder API sets any header explicitly, most commonly Location on a successful create.



Location on Create

`.header(HttpHeaders.LOCATION, "/api/v1/employees/" + id)`
tells the client exactly where the new resource lives.



Cache-Control When Needed

`.header(HttpHeaders.CACHE_CONTROL, "max-age=300")`
declares the caching intent from Module 13.



Fluent Builder Chain

Multiple `.header()` calls chain naturally before the final `.body(...)` call.

KEY TAKEAWAY A 201 without a Location header is technically valid HTTP but incomplete REST -- always pair them.



SETTING RESPONSE HEADERS

Response headers provide metadata about the response. In Spring, you can set headers using **ResponseEntity**, **HttpHeaders**, or **@ResponseHeader**.

Key Points



Headers contain metadata such as content type, caching, security, location, etc.



You can add, set, or remove headers in Spring responses.



Useful for caching, pagination, file downloads, CORS, and security policies.



Use **ResponseEntity**, **HttpHeaders**, or **@ResponseHeader** for header management.

1. Using ResponseEntity with HttpHeaders

```
@GetMapping("/employees/{id}")
public ResponseEntity<Employee> getEmployee(@PathVariable Long id) {
    Employee emp = employeeService.findById(id);

    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_JSON);
    headers.add("X-Application", "Employee Service");
    headers.add("Cache-Control", "no-store");

    return new ResponseEntity<>(emp, headers, HttpStatus.OK);
}
```

2. Using @ResponseHeader on Controller

```
@GetMapping("/health")
@ResponseHeader(name = "X-Application", value = "Employee Service")
@ResponseHeader(name = "Cache-Control", value = "no-cache")
public String health() {
    return "OK";
}
```

Example HTTP Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store
X-Application: Employee Service
Content-Length: 126
|
{"id":101,"name":"Alice Johnson","email":"alice@example.com"}
```



How Response Headers Help

	Performance	Enable caching and reduce unnecessary network calls.
	Security	Enforce security policies and protect sensitive data.
	Client Behavior	Guide clients on how to handle the response.
	Standards & Interop	Maintain compatibility with HTTP standards and tools.

Common Response Headers

Header	Purpose
Content-Type	Indicates the media type of the response body
Cache-Control	Instructs caching behavior (e.g., no-store, max-age)
Content-Length	Specifies the size of the response body
Location	Provides the URL of the newly created resource
X-Application	Custom header for application identification
ETag	Entity tag for caching and concurrency control
Last-Modified	Timestamp of the last modification
Accept-Ranges	Indicates support for partial content requests

Best Practices

- ✓ Set Content-Type explicitly when returning custom responses.
- ✓ Use Cache-Control appropriately based on data sensitivity.
- ✓ Include Location header after creating a resource (POST).
- ✓ Use custom headers (e.g., X-*) for application metadata.
- ✓ Avoid exposing sensitive information in headers.



Tip: Headers are part of your API contract. Document them in OpenAPI for better client integration.



Remember: Well-designed headers improve performance, security, and client experience.

CONTENT NEGOTIATION AND JSON MEDIA TYPES

Spring picks a response format by matching the client's Accept header against what the controller can produce.



Negotiation, Not Assumption

Spring compares the client's Accept header against each mapping's declared produces types.



produces = "application/json"

@GetMapping(produces = MediaType.APPLICATION_JSON_VALUE) makes the supported format explicit.



406 on Mismatch

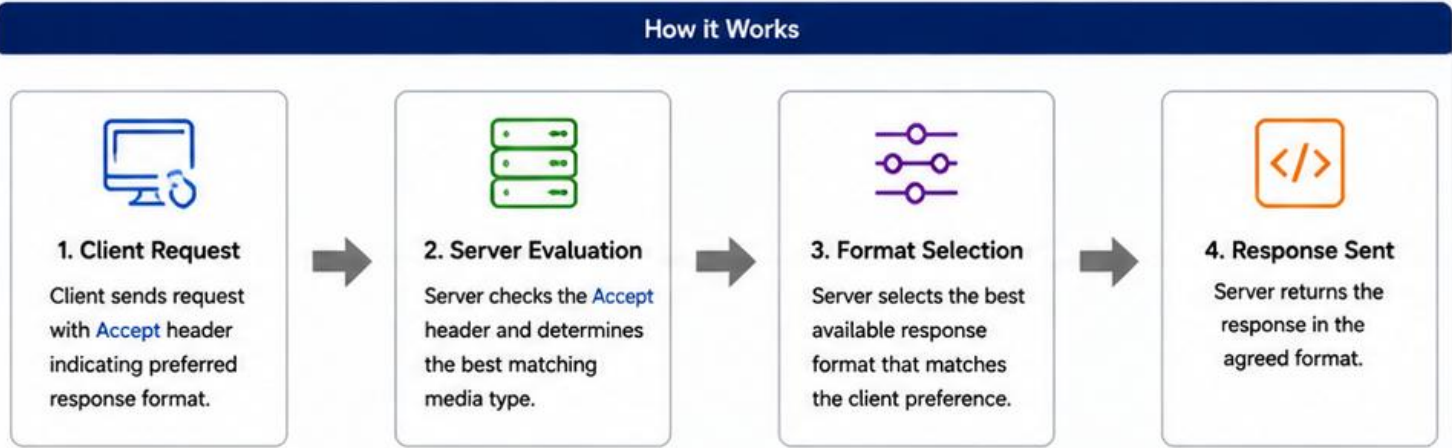
An Accept header the controller can't satisfy returns 406 Not Acceptable, not a silently wrong format.

KEY TAKEAWAY This API produces only application/json -- content negotiation exists mainly so a mismatched Accept header fails clearly.

CONTENT NEGOTIATION



Content negotiation is the process where the server and client agree on the best format to use for the response. In Spring, this is based on the **Accept** request header.



Key Points

- ✓ Client specifies preferred format using the **Accept** header.
- ✓ Server returns the response in the best matching format.
- ✓ If no match is found, server may return **466 Not Acceptable**.
- ✓ Spring MVC handles content negotiation automatically.
- ✓ Supports multiple formats: JSON, XML, HTML, etc.

Example Controller in Spring

```
@GetMapping(value = "/employees/{id}", produces = {
    MediaType.APPLICATION_JSON_VALUE,
    MediaType.APPLICATION_XML_VALUE
})
public ResponseEntity<Employee> getEmployee(@PathVariable Long id) {
    Employee emp = employeeService.findById(id);
    return ResponseEntity.ok(emp);
}
```

Negotiation Process		
Step	Description	Example
1. Client Request	Client sends request with Accept header.	Accept: application/json
2. Server Checks	Server checks available media types.	application/json, application/xml
3. Match Found	Server finds the best match.	JSON is selected
4. Response Sent	Response is returned in the selected format.	Content-Type: application/json

Tip: Use the **produces** attribute in **@RequestMapping** to declare supported media types and control content negotiation behavior.

Common Media Types	
Media Type	Description
application/json	JSON format
application/xml	XML format
text/html	HTML format
text/plain	Plain text
application/octet-stream	Binary data (file download)

Example Requests and Responses

Request

GET /employees/101
Accept: application/json

Response

HTTP/1.1 200 OK
Content-Type: application/json

```
{
  "id": 101,
  "name": "Alice Johnson",
  "email": "alice@example.com"
}
```

JSON MEDIA TYPES



JSON media types are used in HTTP requests and responses to indicate that the message body contains JSON data.

Key Points

Media type is carried in **Content-Type** (request) and **Accept** (response preference) headers.

application/json is the standard media type for JSON data.

application/problem+json is used for error responses (RFC 7807).

Use **UTF-8** encoding for JSON data.

Common JSON Media Types		
Media Type	Usage	Example
application/json	Standard JSON data	Content-Type: application/json Accept: application/json
application/problem+json	Problem Details for HTTP APIs (errors)	Content-Type: application/problem+json
application/ld+json	JSON-LD for linked data	Content-Type: application/ld+json
application/json; charset=utf-8	JSON with explicit UTF-8 charset	Content-Type: application/json; charset=utf-8
application/vnd.api+json	JSON:API specification	Content-Type: application/vnd.api+json

Where JSON Media Types Are Used

In Requests (Client → Server)

- Content-Type: **application/json**
Indicates the body format of POST, PUT, PATCH requests.

In Responses (Server → Client)

- Content-Type: **application/json**
Indicates the response body format.
- Influences how the client parses the data.

Example: Request with JSON

```
POST /employees HTTP/1.1
Host: api.example.com
Content-Type: application/json
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
{
  "name": "Alice Johnson",
  "email": "alice@example.com",
  "salary": 85000.00
}
```



Example: Response with JSON

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
{
  "id": 101,
  "name": "Alice Johnson",
  "email": "alice@example.com",
  "salary": 85000.00,
  "hireDate": "2024-01-15"
}
```

Example: Problem Details (application/problem+json)

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json
{
  "type": "https://example.com/problems/validation-error",
  "title": "Validation Error",
  "status": 400,
  "detail": "Email must be a valid email address.",
  "instance": "/employees",
  "invalid-params": [
    { "name": "email", "reason": "Invalid format" }
  ]
}
```



Tip: Always set **Content-Type** and **Accept** headers correctly to ensure proper communication and avoid **415 Unsupported Media Type** errors.



Best Practice: Use **application/json** with **UTF-8** encoding for all JSON payloads.

TRY IT YOURSELF — EXERCISE 3: SHAPE RESPONSES WITH RESPONSEENTITY

Reinforces: ResponseEntity, status codes in Spring, response headers, and content negotiation as you just covered them.

STEPS (notes/lab24-response-map.md)

Step	What To Do
1 — Return Table	For each handler, write the ResponseEntity it returns on success.
2 — Location Header	Show the Location header the create endpoint sets for CUS-1003.
3 — Not Found	Say how getOne turns a missing CUS-9999 into a 404 without a null body.
4 — Media Type	Name the media type produced and where it is declared.

ResponseEntity shapes

```
getOne    -> ResponseEntity.ok(dto) | ResponseEntity.notFound().build()
create    -> ResponseEntity.created(URI /api/customers/CUS-1003).body(dto)
produces  = application/json declared on the class
```

TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-response-entity-map.md (workspace: examples/module-24-exercises).

CONTROLLER → SERVICE FLOW

A controller method's body is almost always one line: delegate, and return what comes back.



Constructor Injection

The EmployeeService is injected into the controller via its constructor, wired automatically by Spring.



One-Line Delegation

```
return  
employeeService.findById(id); --  
the controller does not touch  
the repository or database  
directly.
```



Service Owns the Rules

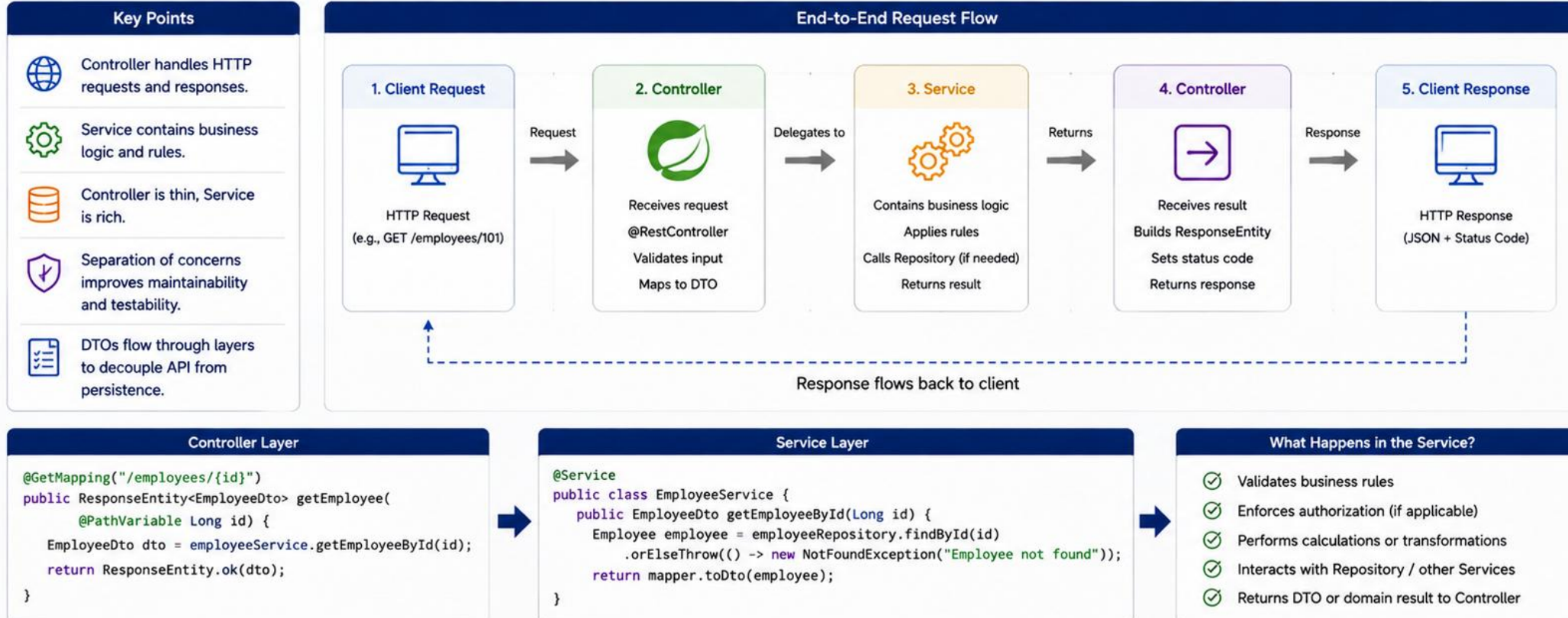
Business rules, transactions, and orchestration all live in the service layer, never the controller.

KEY TAKEAWAY If you can't describe a controller method's body in one sentence, business logic has crept in where it doesn't belong.

CONTROLLER → SERVICE FLOW



In a Spring Boot REST API, the **Controller** handles the HTTP request, validates input, and delegates business logic to the **Service** layer. The **Service** processes the request and returns the result back to the Controller.



Tip: Keep controllers thin and delegate business logic to services. This leads to cleaner code, easier testing, and better scalability.

DTO INTEGRATION

Controllers speak exclusively in DTOs -- entities never cross the controller boundary in either direction.



Request DTOs In

EmployeeCreateRequest and EmployeeUpdateRequest carry exactly what a client can legitimately send.



Response DTOs Out

EmployeeResponse carries exactly what a client should see -- never more, never less.



Service Layer Maps Between

The service converts DTOs to/from entities; the controller never sees an entity at all.

KEY TAKEAWAY DTOs at the controller boundary are what let the database schema and the API contract evolve independently.

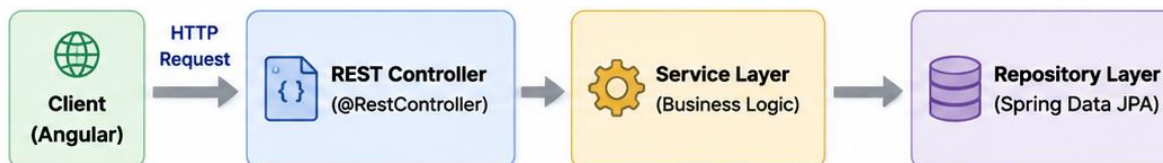
What is a DTO?

A **DTO (Data Transfer Object)** is a simple object used to transfer data between layers or across system boundaries.

Why Use DTOs?

- ✓ Decouples API layer from domain entities
- ✓ Controls what data is exposed
- ✓ Supports validation and transformation
- ✓ Improves security and performance
- ✓ Provides a stable API contract

How DTOs Fit in the Controller → Service → Repository Flow



DTOs Act as the Data Contract

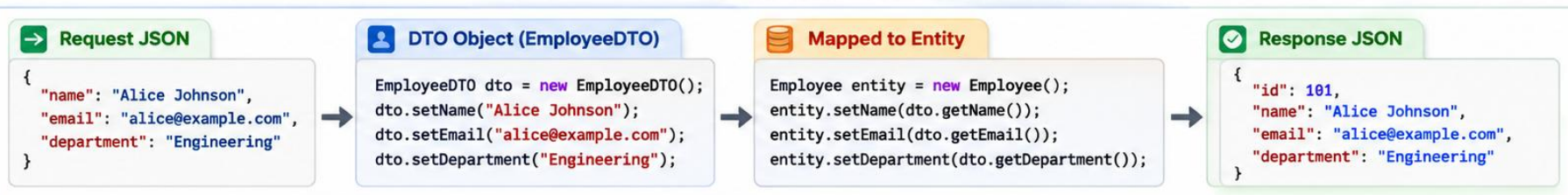
- ✓ Input DTOs: Received from client and validated
- ✓ Output DTOs: Sent back to client
- ✓ Entities stay internal to the system



Example: EmployeeDTO.java

```

public class EmployeeDTO {
    private Long id;
    private String name;
    private String email;
    private String department;
    // getters and setters
}
  
```



Best Practices

Use DTOs for external APIs

Never expose entities directly

Keep DTOs simple and purpose-specific

Validate DTOs at the controller boundary

Evolve DTOs carefully without breaking clients

★ **Key Takeaway:** DTOs create a clean contract between layers, improve security, and give you full control over what your API exposes.

VALIDATION AND HANDLING INVALID REQUESTS

@Valid triggers Bean Validation on a request DTO before the controller method body ever runs.



@Valid Triggers Checking

Add @Valid before @RequestBody on the create method, and validation runs automatically.



Constraints on the DTO

@NotBlank, @Email, and @Positive on EmployeeCreateRequest's fields declare the rules once.



400 Before the Method Runs

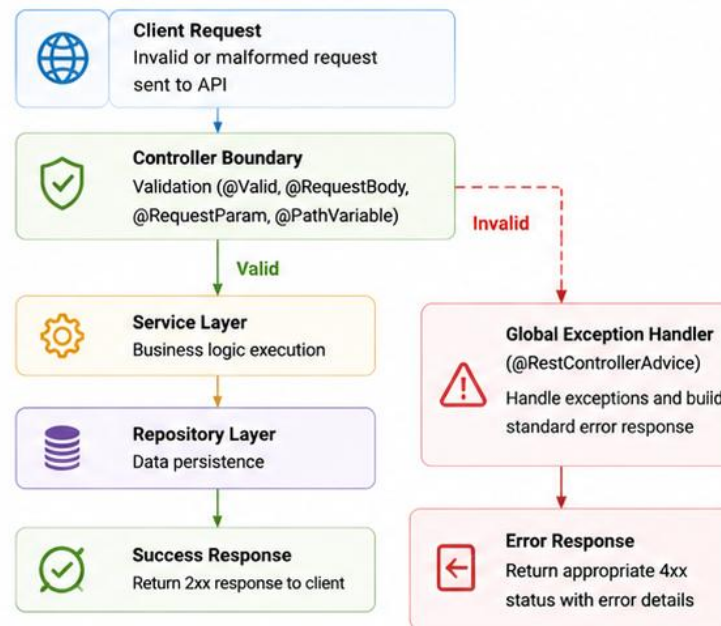
A failed validation short-circuits the request -- the controller method body never executes at all.

KEY TAKEAWAY @Valid plus constraint annotations on the DTO turns validation into a declarative, one-word addition, not hand-written if-checks.

HANDLING INVALID REQUESTS

Detect invalid input at the controller boundary and return meaningful, consistent error responses to the client.

INVALID REQUEST HANDLING FLOW



COMMON CAUSES OF INVALID REQUESTS



Validation Failures

Required fields missing, invalid formats, or constraint violations.



Malformed JSON

Invalid JSON structure or unsupported data types.



Unsupported Operations

Unexpected HTTP method or unsupported content type.



Authentication / Authorization

Missing, invalid, or expired credentials.



Invalid Resource Reference

Resource not found or invalid path parameters.

RELEVANT HTTP STATUS CODES

400	Bad Request	Malformed request or validation failure
401	Unauthorized	Authentication required or failed
403	Forbidden	Authenticated but not authorized
404	Not Found	Resource does not exist
415	Unsupported Media Type	Invalid or unsupported content type
422	Unprocessable Entity	Semantically invalid request data

EXAMPLE: INVALID REQUEST RESPONSE

```

{
  "timestamp": "2025-05-18T10:15:30Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed",
  "path": "/api/employees",
  "errors": [
    {
      "field": "email",
      "message": "Invalid email format"
    },
    {
      "field": "salary",
      "message": "Salary must be greater than 0"
    }
  ]
}

```

COMMON VALIDATION ANNOTATIONS

@NotNull	Field must not be null
@NotBlank	String must not be null or empty
@Size(min, max)	Length / size must be within range
@Email	Must be a valid email format
@Min(value)	Numeric value must be >= given value
@Max(value)	Numeric value must be <= given value
@Pattern(regex)	String must match regex pattern

BEST PRACTICES



Validate at the controller boundary using @Valid and Bean Validation.



Return meaningful, client-friendly error messages.



Use a consistent error response structure across all APIs.



Include a unique error code for easier client-side handling.



Never expose internal stack traces or sensitive system information.

CONTROLLER EXCEPTION FLOW

@ControllerAdvice centralizes every controller's error handling into one place, matching Module 13's error shape.



Centralized, Not Repeated

@ControllerAdvice applies its exception handling across every controller, not just one.



@ExceptionHandler Per Type

One method per exception type
-- EmployeeNotFoundException maps to 404, validation failures map to 400.



Same JSON Shape Every Time

Every handler returns the exact timestamp/status/error/message/path body from Module 13.

KEY TAKEAWAY One @ControllerAdvice class, a handful of @ExceptionHandler methods, and every controller in the API returns errors identically.

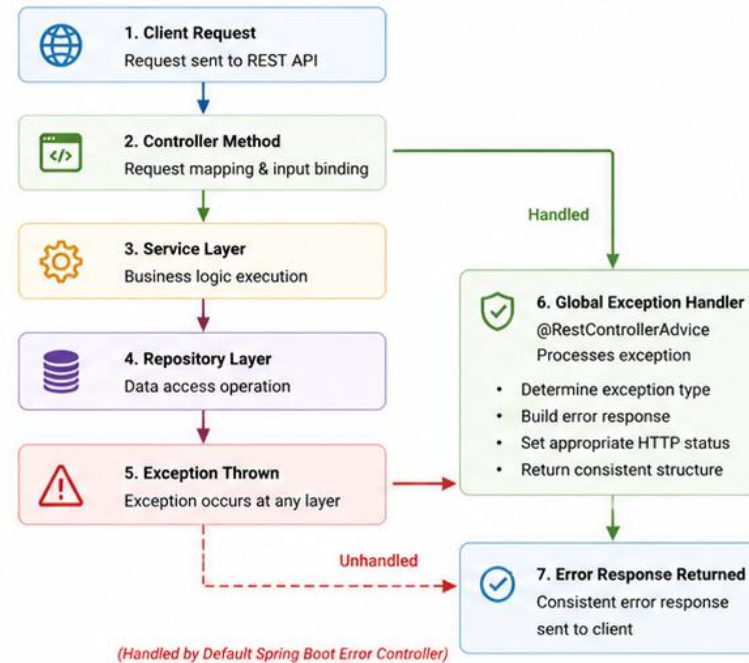
CONTROLLER EXCEPTION FLOW

Exceptions can occur at different layers while processing a request.
A centralized exception handling strategy ensures consistent, meaningful responses to clients.

WHERE EXCEPTIONS CAN OCCUR



EXCEPTION FLOW IN SPRING BOOT



GLOBAL EXCEPTION HANDLER EXAMPLE

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
        ResourceNotFoundException ex) {
        ErrorResponse error = ErrorResponse.builder()
            .timestamp(Instant.now())
            .status(HttpStatus.NOT_FOUND.value())
            .error("Not Found")
            .message(ex.getMessage())
            .path(getCurrentPath())
            .build();
        return new ResponseEntity<>(error,
            HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(ConstraintViolationException.class)
    public ResponseEntity<ErrorResponse> handleConstraint(
        ConstraintViolationException ex) {
        // Build and return 400 Bad Request response
    }
}
```

COMMON EXCEPTIONS AND HTTP STATUS

Exception	Description	HTTP Status
MethodArgumentNotValidException	Validation failed on request body	400 Bad Request
MethodArgumentTypeMismatchException	Invalid path variable / parameter	400 Bad Request
ResourceNotFoundException	Requested resource not found	404 Not Found
ConstraintViolationException	DB constraint or integrity violation	409 Conflict
AccessDeniedException	User not authorized	403 Forbidden
InternalServerErrorException	Unexpected server error	500 Internal Server Error

EXCEPTION HANDLING BEST PRACTICES



Use @RestControllerAdvice for centralized handling.



Return consistent error response structure.



Log full exception details internally.



Do not expose internal stack traces to clients.



Use meaningful messages and correct status codes.

BENEFITS



Consistent client experience



Easier debugging and monitoring



Improved security (no info leakage)



Better maintainability and scalability

TRY IT YOURSELF — EXERCISE 4: TRACE DTO, VALIDATION AND ERRORS

Reinforces: the controller-to-service flow, DTO integration, validation, and the controller exception path you just covered.

STEPS (notes/lab24-validation-flow.md)

Step	What To Do
1 — Layer Boundary	Say what the controller does and what it must delegate to the service.
2 — DTO vs Entity	Give one reason the API exposes a DTO rather than the JPA entity.
3 — Validation Trigger	Name the annotation that triggers validation and the status a failure returns.
4 — Exception Path	Trace a not-found exception from the service to the client response.

Controller responsibilities

```
controller : bind, validate (@Valid), delegate, map result -> status
service    : business rules, transactions, persistence orchestration
@Valid fails -> MethodArgumentNotValidException -> 400 error envelope
```

TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-dto-validation-flow.md (workspace: examples/module-24-exercises).

REST ENDPOINT NAMING STANDARDS

Module 13's URI rules, restated as coding standards for every controller mapping written from here on.

Standard	Applied In Code
Plural nouns, no verbs	@RequestMapping("/api/v1/employees"), never "/getEmployees"
Path variables for identity	@GetMapping("/{employeeId}"), never a query-string id
Consistent casing	camelCase path variable names, lowercase-hyphenated static segments
Version in the base path	"/api/v1/employees" at the class level, applied to every method

KEY TAKEAWAY Every mapping annotation's path string is a direct, checkable implementation of a Module 13 URI rule.



REST ENDPOINT NAMING STANDARDS

Consistent naming makes APIs intuitive, predictable, and easy to use.

Well-designed endpoints improve developer experience and long-term maintainability.

CORE PRINCIPLES



Resource-Based

Name endpoints around resources, not actions. Use nouns, not verbs.



Hierarchical

Use a clear resource hierarchy to represent relationships.



Lowercase

Use lowercase letters for all URIs.



Use Hyphens

Use hyphens (-) for readability. Avoid underscores (_).



Plural for Collections

Use plural nouns for resource collections. Use singular for individual resources.



Versioning

Include API version in the URI path. Example: `/api/v1/`

GOOD vs BAD EXAMPLES

GOOD (Recommended)	BAD (Avoid)	WHY?
✓ GET <code>/api/v1/employees</code>	✗ GET <code>/api/v1/getEmployees</code>	Avoid verbs in URIs
✓ GET <code>/api/v1/employees/123</code>	✗ GET <code>/api/v1/employee?id=123</code>	Use path for resource identity, not query
✓ POST <code>/api/v1/employees</code>	✗ POST <code>/api/v1/createEmployee</code>	Use collection URI for creation
✓ PUT <code>/api/v1/employees/123</code>	✗ PUT <code>/api/v1/update/123</code>	Resource URI represents the target
✓ DELETE <code>/api/v1/employees/123</code>	✗ DELETE <code>/api/v1/deleteEmployee/123</code>	Use standard HTTP methods with resource URIs
✓ GET <code>/api/v1/departments/10/employees</code>	✗ GET <code>/api/v1/getEmployeesByDept/10</code>	Use hierarchical resource relationships

RESOURCE NAMING GUIDELINES

- Use meaningful, business-relevant resource names.
- Avoid abbreviations unless they are universally understood (e.g., `id`, `api`).
- Use consistent naming across the entire API.
- Do not include file extensions (e.g., `.json`, `.xml`).
- Avoid trailing slashes.

EXAMPLES

- ✓ `/api/v1/projects`
- ✓ `/api/v1/projects/456`
- ✓ `/api/v1/projects/456/tasks`
- ✗ `/API/V1/Projects/`
- ✗ `/api/v1/getProjectDetails`
- ✗ `/api/v1/projects.json`

COMMON RESOURCE PATTERNS

Collection	GET	<code>/api/v1/employees</code>
Single Resource	GET	<code>/api/v1/employees/{id}</code>
Create	POST	<code>/api/v1/employees</code>
Update (Replace)	PUT	<code>/api/v1/employees/{id}</code>
Update (Partial)	PATCH	<code>/api/v1/employees/{id}</code>
Delete	DELETE	<code>/api/v1/employees/{id}</code>
Nested Resource	GET	<code>/api/v1/departments/{id}/employees</code>
Filter / Search	GET	<code>/api/v1/employees?status=active</code>
Pagination	GET	<code>/api/v1/employees?page=1&size=20</code>
Sort	GET	<code>/api/v1/employees?sort=lastName,asc</code>

BENEFITS



Improves consistency and usability



Simplifies documentation and discoverability



Enhances collaboration across teams



Reduces maintenance and integration issues



BEST PRACTICES SUMMARY



Think resources, not actions.



Keep URIs short, clean, and intuitive.



Follow HTTP standards consistently.



Design for clarity today, scalability tomorrow.

DESIGNING COLLECTION ENDPOINTS

The collection endpoint -- GET and POST on the bare resource path -- carries the most traffic in most APIs.



GET Lists, POST Creates

@GetMapping and @PostMapping, both with no path variable, share the bare /employees mapping.



Return a Page, Not a List

A paginated response wraps the array with total counts, so clients never guess at size.



Never Unbounded

A collection endpoint with no pagination will eventually return every employee in one massive response.

KEY TAKEAWAY A collection endpoint returning an unbounded, unpaginated list is a production incident waiting to happen.

DESIGNING COLLECTION ENDPOINTS

Collection endpoints manage groups of resources. They enable clients to list, filter, sort, and paginate data consistently and efficiently.

COLLECTION ENDPOINT PRINCIPLES



Use Plural Nouns

Collections represent resource groups, not single items.



Support Query Parameters

Enable filtering, sorting, pagination, and searching.



Return Consistent Responses

Use a standard structure for data and metadata.



Limit and Paginate

Protect performance by limiting result set size.



Do Not Overload Endpoints

Use query parameters, not multiple endpoints for every combination.

URI DESIGN OPTIONS



GOOD

- GET /api/v1/employees
- GET /api/v1/employees?page=0&size=20
- GET /api/v1/employees?status=active&sort=lastName,asc



AVOID

- GET /api/v1/getEmployees
- GET /api/v1/employeeList
- GET /api/v1/employees/getAll

ENDPOINT PATTERN

GET

/api/v1/employees

Retrieve a collection of employees

COMMON QUERY PARAMETERS

Parameter	Description	Example
page	Page number (0-based)	page=0
size	Page size (number of items)	size=20
sort	Sort field and direction	sort=lastName,asc
filter	Filter by field value	status=active
search	Search by keyword	search=john

EXAMPLE REQUESTS

```
GET /api/v1/employees
GET /api/v1/employees?page=0&size=20
GET /api/v1/employees?sort=lastName,asc
GET /api/v1/employees?status=active
GET /api/v1/employees?search=john&page=0&size=10&sort=hireDate,desc
```

EXAMPLE RESPONSE

```
{
  "data": [
    {
      "id": 101,
      "firstName": "John",
      "lastName": "Doe",
      "email": "john.doe@example.com",
      "status": "active"
    },
    {
      "id": 102,
      "firstName": "Jane",
      "lastName": "Smith",
      "email": "jane.smith@example.com",
      "status": "active"
    }
  ],
  "meta": {
    "page": 0,
    "size": 20,
    "totalElements": 245,
    "totalPages": 13,
    "sort": "lastName,asc"
  },
  "links": {
    "self": "/api/v1/employees?page=0&size=20&sort=lastName,asc",
    "next": "/api/v1/employees?page=1&size=20&sort=lastName,asc",
    "prev": null,
    "first": "/api/v1/employees?page=0&size=20&sort=lastName,asc",
    "last": "/api/v1/employees?page=12&size=20&sort=lastName,asc"
  }
}
```

RESPONSE STRUCTURE BEST PRACTICES



Wrap data in a "data" field

Keeps structure extensible.



Include pagination metadata

Helps clients navigate result sets.



Provide HATEOAS-style links

Improves discoverability and navigation.



Maintain consistency

Follow the same structure across all collections.

COLLECTION ENDPOINT CHECKLIST

- ✓ Use plural nouns for collection URIs.
- ✓ Support pagination with page and size.
- ✓ Support sorting with sort parameter.
- ✓ Support filtering and search as needed.
- ✓ Return consistent response structure.
- ✓ Include metadata and navigation links.
- ✓ Limit maximum page size.
- ✓ Document query parameters in OpenAPI.

PAGINATION, FILTERING, AND SORTING ENDPOINTS

Spring Data's Pageable parameter can absorb page, size, and sort in one bound object.



Pagination via Pageable

public
Page<EmployeeResponse>
list(Pageable pageable) binds
?page=&size=&sort=
automatically.



Filtering via @RequestParam

Individual filter fields like
department stay as their own
explicit @RequestParam values
alongside Pageable.



Sorting Folded Into Pageable

?sort=lastName,asc is parsed by
Spring Data into a Sort object
inside Pageable -- no manual
parsing needed.

KEY TAKEAWAY Pageable collapses page/size/sort into one Spring-managed parameter, instead of three separate @RequestParam values.

PAGINATION ENDPOINTS

Pagination endpoints return large datasets in smaller, manageable chunks. They improve performance, reduce payload size, and enhance user experience.

WHY PAGINATION?



Performance

Load only the required data, reduce response time and memory usage.



Scalability

Efficiently handle large datasets as data grows.



Better UX

Faster page rendering and smoother navigation.



Bandwidth Optimization

Transfer only what the client needs.



Consistent Navigation

Standard metadata helps clients navigate reliably.

PAGINATION ENDPOINT PATTERN

GET /api/v1/employees
Retrieve a paginated list of employees

COMMON QUERY PARAMETERS

Parameter	Type	Description	Default	Example
page	int	Page number (0-based)	0	page=0
size	int	Page size (number of items)	20	size=20
sort	string	Sort field and direction	id,asc	sort=lastName,desc
filter	string	Filter by field value	-	status=active
search	string	Search by keyword	-	search=john

EXAMPLE REQUESTS

```
GET /api/v1/employees?page=0&size=20 # First page, 20 items
GET /api/v1/employees?page=2&size=20 # Third page
GET /api/v1/employees?size=50&sort=lastName,asc # Sort by lastName ASC
GET /api/v1/employees?status=active&page=0&size=10 # Filter and paginate
GET /api/v1/employees?search=john&page=0&size=20 # Search and paginate
```

EXAMPLE RESPONSE (SPRING DATA PAGE)

```
{
  "content": [
    { "id": 101,
      "id": "101", "firstName": "lastName": "Doe", "email": "john.doe@example.com" },
    { "id": 102,
      "id": "102", "firstName": "lastName": "Smith", "email": "jane.smith@example.com" }
  ],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 20,
    "sort": [{ "property": "lastName", "direction": "DESC" }],
    "offset": 0,
    "paged": true,
    "unpaged": false
  },
  "totalElements": 245,
  "totalPages": 13,
  "last": false,
  "size": 20,
  "number": 0,
  "numberOfElements": 20,
  "first": true,
  "sort": { "sorted": true, "unsorted": false, "empty": false }
}
```

NAVIGATION FLOW



RECOMMENDED RESPONSE FIELDS



BEST PRACTICES

- ✓ Use 0-based page index for consistency.
- ✓ Set sensible default page size (e.g., 20 or 50).
- ✓ Enforce maximum page size to prevent abuse.
- ✓ Support sorting and filtering along with pagination.
- ✓ Return consistent metadata structure.
- ✓ Use Spring Data Page<T> for standardization.
- ✓ Document parameters in OpenAPI.

FILTERING ENDPOINTS

Filtering endpoints allow clients to retrieve only the data that matches specific criteria. They reduce payload size, improve performance, and enhance usability.

WHY FILTERING?



Focus on Relevant Data

Return only the records that match user-specified criteria.



Better Performance

Reduce unnecessary data transfer and processing.



Improved User Experience

Enable precise search and discovery of information.



Combine with Pagination & Sorting

Work together to provide powerful data exploration.



Standardized Queries

Consistent filtering conventions simplify API usage.

FILTERING ENDPOINT PATTERN

GET`/api/v1/employees`

Retrieve employees with optional filters

COMMON FILTER PARAMETERS

Parameter	Type	Description	Example
status	string	Filter by employment status	status=active
departmentId	long	Filter by department	departmentId=10
role	string	Filter by job role	role=developer
fromDate	string (date)	Filter by hire date from	fromDate=2024-01-01
toDate	string (date)	Filter by hire date to	toDate=2024-12-31
minSalary	decimal	Filter by minimum salary	minSalary=50000
maxSalary	decimal	Filter by maximum salary	maxSalary=120000
search	string	Search by keyword (name/email)	search=john

EXAMPLE FILTERED REQUESTS

```
GET /api/v1/employees?status=active           # Active employees only
GET /api/v1/employees?departmentId=10         # Employees in department 10
GET /api/v1/employees?role=developer&status=active # Multiple filters
GET /api/v1/employees?minSalary=50000&maxSalary=100000 # Salary range
GET /api/v1/employees?fromDate=2024-01-01&toDate=2024-12-31 # Hire date range
GET /api/v1/employees?search=john&status=active&page=0&size=20 # Search + filter + paginate
```

EXAMPLE RESPONSE

```
{
  "content": [
    { "id": 101, "firstName": "John", "lastName": "Doe",
      "email": "john.doe@example.com", "status": "active" },
    { "id": 105, "firstName": "Jane", "lastName": "Smith",
      "email": "jane.smith@example.com", "status": "active" }
  ],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 20,
    "sort": [ { "property": "lastName", "direction": "ASC" } ]
  },
  "totalElements": 35,
  "totalPages": 2,
  "last": false,
  "size": 20,
  "number": 0,
  "numberOfElements": 20,
  "first": true,
  "sort": { "sorted": true, "unsorted": false, "empty": false }
}
```

FILTERING STRATEGIES



Query Parameters
Most common and easy to implement.



Multiple Parameters
Combine parameters for complex filters.



Search Parameter
Free-text search across fields.



Date Range Filters
Use fromDate and toDate pattern.



Range Filters
Use min / max for numeric data.

RECOMMENDED RESPONSE FIELDS



content
Filtered list of items



totalElements
Total matching records



totalPages
Total pages available



pageable
Pagination and sort metadata



first / last
Navigation indicators

BEST PRACTICES

- ✓ Use clear and consistent parameter names.
- ✓ Support multiple filters through query parameters.
- ✓ Combine filtering with sorting and pagination.
- ✓ Validate filter values and handle invalid input.
- ✓ Document all filter parameters in OpenAPI.
- ✓ Avoid exposing internal database fields.
- ✓ Set reasonable defaults when filters are missing.
- ✓ Monitor and optimize performance of filtered queries.



Sorting Endpoints



Sorting allows API consumers to control the order of returned data using query parameters.

EXAMPLE ENDPOINT

GET `/api/employees?sort=lastName,asc`

The **sort** parameter follows this format:

`sort=<fieldName>,<direction>`

Direction: **asc** (ascending) or
desc (descending)

SORTING EXAMPLES

Example Request	Description	Result
<code>GET /api/employees?sort=lastName,asc</code>	Sort by lastName ascending (A-Z)	 A → Z
<code>GET /api/employees?sort=lastName,desc</code>	Sort by lastName descending (Z-A)	 Z → A
<code>GET /api/employees?sort=department,asc</code>	Sort by department ascending	 A → Z
<code>GET /api/employees?sort=hireDate,desc</code>	Sort by hireDate descending (newest first)	 New → Old
<code>GET /api/employees? sort=lastName,asc&sort=firstName,asc</code>	Sort by lastName, then firstName (both ascending)	 1 1 2 1 Z



TIP: Multiple sort parameters can be used for multi-level sorting.

★ KEY POINTS



Sorting gives clients control over the order of results, making responses more relevant and user-friendly.



Use the **sort** query parameter to specify the field and direction.



Supported directions are **asc** (ascending) and **desc** (descending).



Multiple sort parameters allow multi-level sorting. The order of parameters defines priority.



Validate and whitelist sortable fields to prevent invalid or unsafe sorting.



Spring Data JPA Example

`GET /api/employees?sort=lastName,asc`

Handled automatically by Spring Data JPA using **Sort** and **Pageable**.

CORS OVERVIEW

Browsers block cross-origin requests by default -- CORS is the server's explicit permission to allow them.



Same-Origin by Default

A browser blocks JavaScript from calling an API on a different origin unless that API explicitly allows it.



Preflight Requests

The browser sends an OPTIONS request first, checking permissions before the real request goes out.



The Server Grants Access

Access-Control-Allow-Origin and related headers tell the browser which origins are trusted.

KEY TAKEAWAY CORS is enforced by the browser, not the server -- the server's job is only to say which origins it trusts.



CORS OVERVIEW



Module 24 REST Endpoints and Request Mapping

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that controls how web applications running on one origin can access resources from a different origin.

WHY CORS MATTERS

- Browsers enforce **Same-Origin Policy** by default.
- CORS allows controlled access to APIs hosted on different domains.
- Essential for frontend applications (e.g., Angular) calling backend APIs (e.g., Spring Boot) on another origin.
- Helps protect users from malicious cross-site requests.



HOW CORS WORKS

Front-End Application
Origin A



<https://app.example.com>

1. Actual Request
(e.g., GET /api/employees)



2. Response
+ CORS Headers
(e.g., Access-Control-Allow-Origin)



API Server
Origin B



<https://api.example.com>

(For non-simple requests)
3. Preflight Request
(OPTIONS)



4. Preflight Response
(Access-Control-Allow-*)



Preflight is triggered for:

- Non-GET/POST/HEAD requests
- Custom headers
- Content-Type other than application/x-www-form-urlencoded, multipart/form-data, text/plain

SIMPLE VS NON-SIMPLE REQUESTS

Simple Requests (No Preflight)

- Methods: GET, POST, HEAD
- Headers: Accept, Accept-Language, Content-Language, Content-Type (with allowed values)
- No preflight OPTIONS request

Non-Simple Requests (Preflight)

- Other HTTP methods (PUT, DELETE, PATCH)
- Custom headers
- Content-Type: application/json, etc.
- Triggers OPTIONS preflight request

KEY CORS HEADERS

Header	Purpose
Access-Control-Allow-Origin	Specifies which origins are allowed
Access-Control-Allow-Methods	Specifies allowed HTTP methods
Access-Control-Allow-Headers	Specifies allowed request headers
Access-Control-Allow-Credentials	Allows cookies/credentials to be sent
Access-Control-Max-Age	Specifies how long preflight response can be cached (in seconds)

BEST PRACTICES



Allow only trusted origins.



Avoid using * when credentials are involved.



Use [Access-Control-Max-Age](#) to reduce preflight requests and improve performance.



Configure CORS on the server, not in the client.



Review and update CORS policies regularly.



TIP: CORS is not a security mechanism for the server. Always implement proper authentication and authorization for your APIs.

ANGULAR-TO-SPRING REST COMMUNICATION

Configuring CORS so the Angular dev server's origin is explicitly trusted by the Spring Boot API.



@CrossOrigin, Per Controller

@CrossOrigin(origins = "http://localhost:4200") on a controller class allows just that origin.



Global CORS Configuration

A CorsConfigurationSource bean applies one consistent policy across every controller in the API.

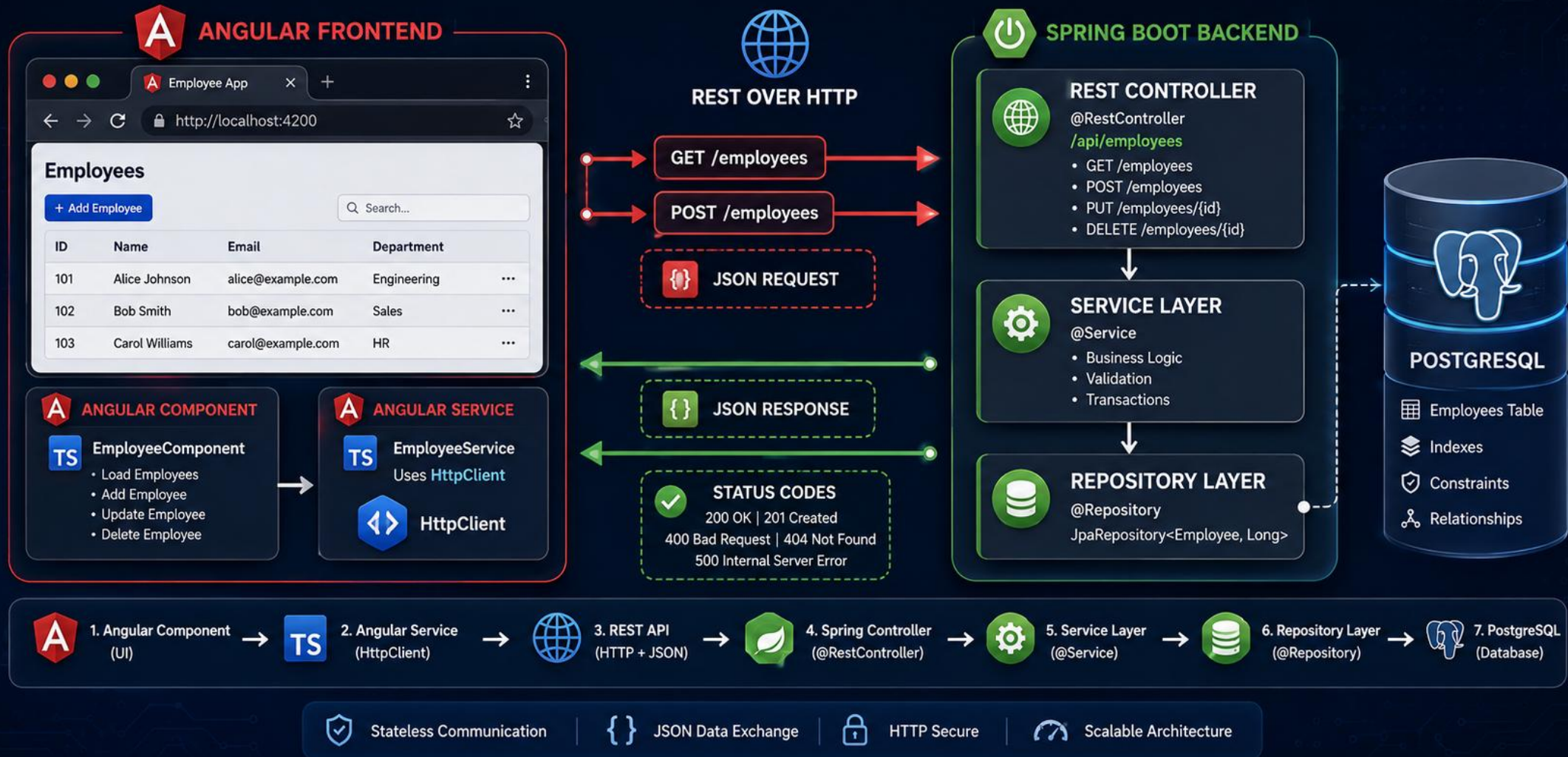


Credentials Need Explicit Origins

Sending cookies or auth headers cross-origin requires a named origin -- a wildcard '*' will not work.

KEY TAKEAWAY @CrossOrigin or a global CorsConfigurationSource -- either way, the allowed origin list must name the Angular app explicitly.

Angular-to-Spring REST Communication



TRY IT YOURSELF — EXERCISE 5: WRITE THE ANGULAR CORS NOTE

Reinforces: the endpoint-naming, collection, CORS, and Angular-to-Spring communication slides you just covered, written as the note Lab 24 grades.

STEPS (notes/lab24-cors-angular.md)

Step	What To Do
1 — Why It Happens	Say in one sentence why the browser blocks the Angular call but curl succeeds.
2 — Allowed Origin	Name the exact dev origin to allow, not a wildcard.
3 — Methods and Headers	List the methods and request headers the CRM UI needs allowed.
4 — Preflight	Say which requests trigger an OPTIONS preflight and why.

CORS for the CRM UI

```
origin http://localhost:4200      (not *, and not with credentials)
methods GET, POST, PUT, DELETE, OPTIONS
headers Content-Type, Authorization, X-Correlation-Id
```

TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-cors-for-angular.md (workspace: examples/module-24-exercises).

REST ENDPOINT SECURITY BOUNDARIES

Every endpoint needs a deliberate answer to 'who is allowed to call this,' not an accidental default.



Authentication vs Authorization

Is the caller who they claim to be (401 if not), and are they allowed to do this specific thing (403 if not)?



Method-Level Rules

@PreAuthorize("hasRole('HR_ADMIN')") on a controller method restricts it to a specific role.



Deny by Default

New endpoints should start restricted and be deliberately opened, never the reverse.

KEY TAKEAWAY An endpoint with no explicit security decision is not neutral -- it's either accidentally open or accidentally blocked.



REST ENDPOINT SECURITY BOUNDARIES



Module 24 REST Endpoints and Request Mapping

Security boundaries protect REST endpoints, data, and operations by enforcing authentication, authorization, validation, and rate limits at every entry point.

SECURITY PRINCIPLES



Authenticate

Verify the identity of every client before granting access.



Authorize

Ensure clients can access only the resources and actions they are permitted to perform.



Validate

Validate all inputs to prevent malicious or invalid data.



Protect

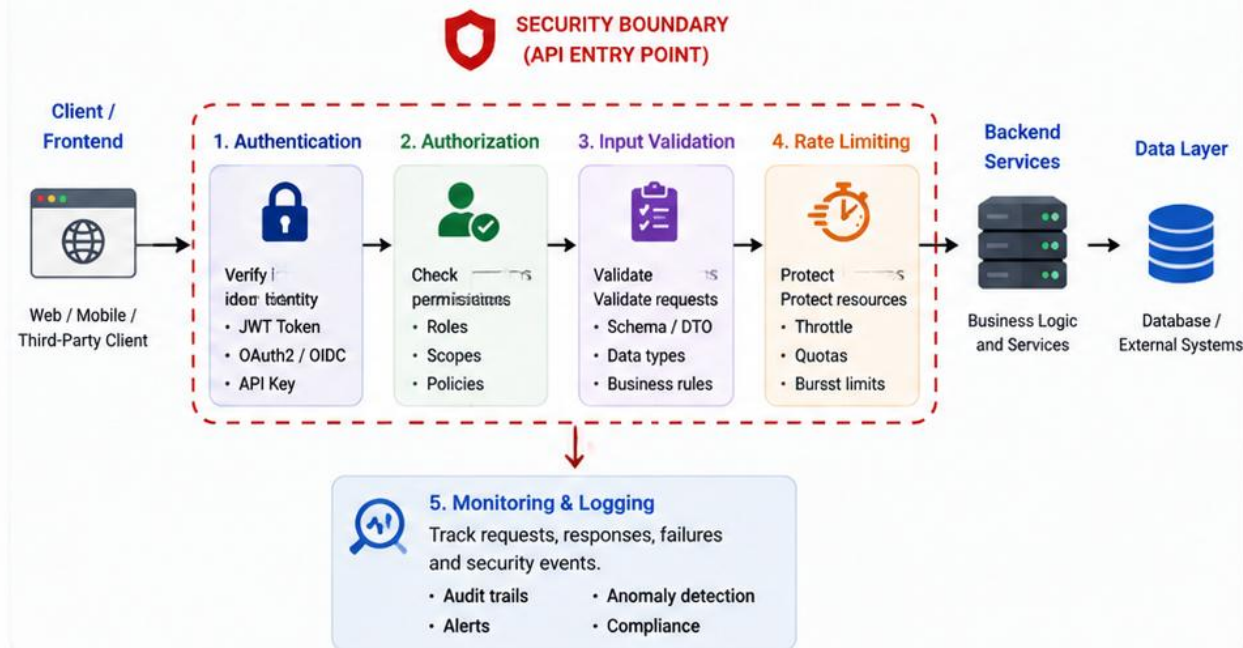
Safeguard data in transit and at rest with encryption and secure configurations.



Monitor & Limit

Monitor usage and apply rate limits to prevent abuse and attacks.

SECURITY BOUNDARIES IN A REST API FLOW



COMMON THREATS



Unauthorized Access

Access without valid authentication.



Broken Authorization

Access to resources without proper permissions.



Injection Attacks

e.g., SQL Injection, XSS via unvalidated input.



Excessive Requests

Abuse of API through high request volume.



Data Exposure

Sensitive data leaked through APIs.

BEST PRACTICES



Use HTTPS everywhere.



Store secrets securely (env vars, vaults).



Apply least privilege access.



Keep dependencies and frameworks up to date.



Review and test security regularly.



TIP: Enforce security at multiple layers – gateway, application, and data – for defense in depth.

TESTING CONTROLLERS: POSTMAN, CURL, AND MOCKMVC

Manual tools for exploring a running API by hand; MockMvc for fast, repeatable automated tests.

Tool	Use It For
cURL	Quick, scriptable command-line requests during manual development
Postman	Saved, organized request collections for exploring and sharing API behavior
MockMvc	Fast, automated tests that don't require a running server

KEY TAKEAWAY Use Postman or cURL to explore by hand while developing; use MockMvc so that behavior stays verified automatically afterward.



TESTING CONTROLLERS WITH POSTMAN/cURL



Module 24 REST Endpoints and Request Mapping

Use Postman or cURL to manually test REST endpoints, validate request/response, headers, status codes, and error handling.

TESTING WITH POSTMAN

GET Send

Params • Authorization Headers (2) Body Pre-request Script Tests Settings Cookies

Query Params

	KEY	VALUE	DESCRIPTION	...	Bulk Edit
☒	includeDetails	true			
☒	fields	id,name,email,department			
	Key	Value	Description		

Body Cookies Headers (5) Test Results 200 OK 158 ms 512 B Save Response

Pretty Raw Preview Visualize JSON ⌵ 🔍

```
1 {
2   "id": 1,
3   "name": "John Doe",
4   "email": "john.doe@example.com",
5   "department": "Engineering"
6 }
7
```

WHAT TO VERIFY

- ✓ Correct HTTP status codes
- ✓ Response body structure
- ✓ Headers (Content-Type, Location, etc.)
- ✓ Validation and error messages
- ✓ Edge cases and invalid inputs

BEST PRACTICES

- ✓ Test all HTTP methods for each endpoint
- ✓ Use environments / variables for base URLs
- ✓ Save requests and create collections
- ✓ Validate both success and error scenarios
- ✓ Document API behavior and sample requests



TIP: Combine manual testing with automated tests (MockMvc) to ensure reliable and repeatable endpoint validation.

TESTING WITH cURL

1. GET – Retrieve an employee

```
curl -X GET "http://localhost:8080/api/employees/1" \
-H "Accept: application/json"
```

2. POST – Create a new employee

```
curl -X POST "http://localhost:8080/api/employees" \
-H "Content-Type: application/json" \
-d '{
  "name": "Jane Smith",
  "email": "jane.smith@example.com",
  "department": "HR"
}'
```

3. PUT – Update an employee

```
curl -X PUT "http://localhost:8080/api/employees/1" \
-H "Content-Type: application/json" \
-d '{"name": "John Doe", "department": "Product"}'
```

4. DELETE – Remove an employee

```
curl -X DELETE "http://localhost:8080/api/employees/1" \
-H "Accept: application/json"
```

EXAMPLE ENDPOINTS

Method	Endpoint	Description
GET	/api/employees	Get all employees
GET	/api/employees/{id}	Get employee by ID
POST	/api/employees	Create a new employee
PUT	/api/employees/{id}	Update an employee
DELETE	/api/employees/{id}	Delete an employee

Adjust base URL
and data as per
your application
configuration.

CONTROLLER TESTING WITH MOCKMVC

MockMvc drives a controller's full request-handling pipeline in a fast, server-free test.



@WebMvcTest, Not @SpringBootTest

A slice test loads only the web layer, mocking the service layer beneath it.



Fluent Request Building

`mockMvc.perform(get("/api/v1/employees/42"))` builds and sends a simulated request.



Fluent Assertions

`.andExpect(status().isOk()).andExpect(jsonPath("$.lastName").value("Nwosu"))` verifies the result.

KEY TAKEAWAY MockMvc tests the same DispatcherServlet pipeline a real request goes through -- without paying for a real server.



CONTROLLER TESTING WITH MOCKMVC

MockMvc allows you to test Spring MVC REST controllers in isolation without starting the full server. It performs requests, validates responses, and ensures correct mapping, status codes, and JSON output.

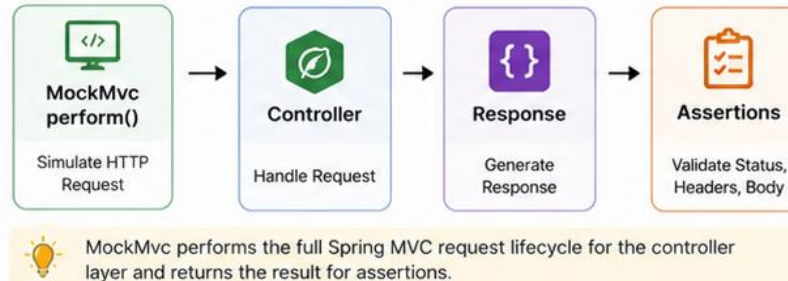


Module 24
REST Endpoints and
Request Mapping

WHY MOCKMVC?

- ✓ Fast and lightweight controller tests
- ✓ No need to start the server
- ✓ Test request mapping, validation, and responses
- ✓ Verify status codes, headers, and JSON payloads
- ✓ Supports security, filters, exception handling
- ✓ Ideal for CI/CD pipelines

HOW MOCKMVC WORKS



TEST ANATOMY

- Request:** HTTP method, URL, params, headers, body
- Perform:** Execute request using MockMvc
- Expect:** Assert status, headers, content, JSON
- Result:** Validate the response

EXAMPLE: CONTROLLER TEST WITH MOCKMVC

```
@WebMvcTest(EmployeeController.class)
class EmployeeControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private EmployeeService employeeService;

    @Test
    void getEmployeeById_shouldReturnEmployee() throws Exception {
        // Arrange
        Employee emp = new Employee(1L, "John Doe", "Engineering");
        when(employeeService.getById(1L)).thenReturn(Optional.of(emp));
        // Act & Assert
        mockMvc.perform(get("/api/employees/1")
            .accept(MediaType.APPLICATION_JSON))

            .andExpect(status().isOk())
            .andExpect(content().contentType(MediaType.APPLICATION_JSON))
            .andExpect(jsonPath("$.id").value(1))
            .andExpect(jsonPath("$.name").value("John Doe"))
            .andExpect(jsonPath("$.department").value("Engineering"));
    }
}
```

COMMON ASSERTIONS

Assertion	What it Verifies
<code>status().isOk()</code>	HTTP 200 OK
<code>status().isCreated()</code>	HTTP 201 Created
<code>status().isBadRequest()</code>	HTTP 400 Bad Request
<code>status().isNotFound()</code>	HTTP 404 Not Found
<code>content().contentType()</code>	Response Content-Type
<code>header().string("X-Total-Count", "5")</code>	Response Header
<code>jsonPath("\$.name").value("John Doe")</code>	JSON field value
<code>jsonPath("\$.items", hasSize(5))</code>	JSON array size

TEST SCENARIOS

- ✓ GET /api/employees/{id} → 200 OK
- ✓ POST /api/employees → 201 Created
- ✓ PUT /api/employees/{id} → 200 OK
- ✓ DELETE /api/employees/{id} → 204 No Content
- ✓ Validation errors → 400 Bad Request
- ✓ Not found → 404 Not Found
- ✓ Server errors → 500 Internal Server Error



TIP

Use `@WebMvcTest` to load only the web layer. Mock dependencies with `@MockBean` for isolated and fast controller tests.



BEST PRACTICE: Test one scenario per test method, use meaningful test names, and validate both success and error responses.

TRY IT YOURSELF — EXERCISE 6: PLAN MOCKMVC TESTS AND SELF-CHECK

Pre-lab gate: design the MockMvc cases Lab 24 grades, then confirm the five earlier notes files are genuinely complete.

STEPS (notes/lab24-mockmvc-readiness.md)

Step	What To Do
1 — Test Matrix	Write four MockMvc cases: get found, get missing, create, list filtered.
2 — Assertions	For each case, name the status and the one JSON path you assert on.
3 — No Real Server	Say why MockMvc needs no running server or browser.
4 — Pass Mark	Mark Pass only if all five earlier notes files are complete, not stubbed.

MockMvc case matrix

```
get CUS-1001 -> 200, jsonPath("$.name").value("Amina Khan")
get CUS-9999 -> 404, no body
create      -> 201, header Location contains /api/customers/
```

TRY IT NOW — Complete Exercise 6 before Lab 24: exercises/exercise-06-mockmvc-readiness.md (workspace: examples/module-24-exercises).

ENTERPRISE REST CONTROLLER BEST PRACTICES

Five practices that keep a growing set of controllers consistent, testable, and safe to change.

Practice	Why It Matters
Keep controllers thin	Business logic stays testable and reusable outside the web layer
Always use ResponseEntity for non-200 outcomes	Status codes are chosen deliberately, never left to defaults
Never return JPA entities	Prevents lazy-loading errors and accidental data leaks
Centralize exception handling	One @ControllerAdvice keeps error responses consistent everywhere
Cover every endpoint with MockMvc	Regressions in status codes or JSON shape are caught automatically

KEY TAKEAWAY Every practice here exists to keep controllers thin, consistent, and safe to change without breaking clients.



ENTERPRISE REST CONTROLLER BEST PRACTICES



Module 24
REST Endpoints and
Request Mapping

Follow consistent patterns to build secure, maintainable, scalable, and consumer-friendly REST APIs that meet enterprise standards.



1 SECURITY FIRST

- Enforce authentication and authorization on every endpoint.
- Use HTTPS everywhere.
- Validate and sanitize all inputs.
- Avoid exposing sensitive data in responses.
- Apply least privilege access.



2 FOLLOW REST CONVENTIONS

- Use resource-based URIs.
- Use correct HTTP methods.
- Use appropriate status codes.
- Keep endpoints predictable and consistent.
- Use plural nouns for collections.



3 KEEP CONTROLLERS FOCUSED

- Keep controllers thin.
- Delegate business logic to services.
- One responsibility per controller.
- Avoid complex logic in request handlers.



4 DESIGN CLEAN APIs

- Use clear and meaningful endpoint names.
- Version your APIs.
- Support pagination, filtering, and sorting.
- Document APIs with OpenAPI.
- Maintain backward compatibility.



5 USE DTOs EFFECTIVELY

- Use DTOs for requests and responses.
- Avoid exposing entity classes.
- Map only required fields.
- Maintain consistency between request and response models.
- Validate DTOs at the boundary.



6 HANDLE ERRORS CONSISTENTLY

- Use a standardized error response structure.
- Return meaningful error messages.
- Use proper HTTP status codes.
- Log errors with sufficient context.
- Do not expose internal details.



7 PERFORMANCE AWARENESS

- Use pagination for large results.
- Avoid N+1 queries in services.
- Use appropriate caching headers.
- Return only necessary data.
- Monitor and optimize slow endpoints.



8 INPUT VALIDATION & BINDING

- Use @Valid and validation annotations.
- Provide clear validation errors.
- Verify path variables and query parameters.
- Enforce constraints at the controller boundary.



9 OBSERVABILITY & MONITORING

- Add correlation IDs to requests.
- Log request and response metadata.
- Track key metrics (latency, errors, throughput).
- Set up alerts for failures and anomalies.



10 TESTABILITY BY DESIGN

- Write unit and integration tests for controllers.
- Use MockMvc for fast tests.
- Test happy paths and edge cases.
- Automate tests in CI/CD.
- Maintain high test coverage.



KEY TAKEAWAY: Well-designed REST controllers improve API reliability, security, performance, and developer experience—delivering long-term value to the enterprise.



GOLDEN RULE



Design for your consumers,
not your convenience.



Secure by default,
not as an afterthought.



Be consistent,
predictable, and simple.



Measure, monitor,
and continuously improve.



Document everything,
test everything.

COMMON CONTROLLER MISTAKES

Five recurring implementation mistakes, and the direct consequence each one causes.

Mistake	Consequence
Business logic inside the controller	Untestable without a full web context; not reusable elsewhere
Returning a plain object instead of <code>ResponseEntity</code>	Every response silently becomes 200, even on partial failure
Returning a JPA entity directly	<code>LazyInitializationException</code> risk, and internal fields leak to clients
<code>try/catch</code> blocks scattered per method	Inconsistent, duplicated error handling instead of one shared shape
No tests beyond manual Postman checks	Regressions in status codes or response shape go unnoticed

KEY TAKEAWAY Almost every controller mistake here is the previous slide's practice, skipped under time pressure -- catch it in review.



COMMON CONTROLLER MISTAKES



Module 24
REST Endpoints and
Request Mapping

Avoid these common mistakes to build correct, maintainable, and enterprise-ready REST controllers.

1 NOT USING PROPER HTTP METHODS



Bad: Using POST for every operation.



Better: Use the correct HTTP method for semantics.

2 IGNORING HTTP STATUS CODES



Bad: Always returning 200 OK.



Better: Return appropriate status codes for success and errors.

3 EXPOSING ENTITY OBJECTS



Bad: Returning JPA entities in responses.



Better: Use DTOs to control the API contract and avoid data leakage.

4 MISSING INPUT VALIDATION



Bad: Not validating request parameters or request body.



Better: Use @Valid and validation annotations.

5 HARDCODING VALUES AND URLS



Bad: Hardcoding IDs, messages, or service URLs.



Better: Use configuration properties and constants.

6 POOR ENDPOINT NAMING



Bad: /getEmpById, /saveEmployee



Better: Use clear, resource-based URIs like /employees/{id}

7 RETURNING INTERNAL ERROR DETAILS



Bad: Returning stack traces or exception messages to clients.



Better: Return a consistent error response without exposing internals.

8 NOT HANDLING EXCEPTIONS GLOBALLY



Bad: Scattering try-catch blocks in every controller.



Better: Use @ControllerAdvice to handle exceptions centrally.

9 IGNORING PAGINATION, FILTERING, SORTING



Bad: Returning large datasets without pagination or filters.



Better: Support pagination, filtering, and sorting consistently.

10 NO API VERSIONING STRATEGY



Bad: Changing existing APIs and breaking clients.



Better: Implement a versioning strategy and maintain backward compatibility.



BEST PRACTICE TAKEAWAY: Design REST controllers with correctness, clarity, consistency, and security in mind.



KEEP IN MIND



Design for your consumers, not your convenience.



Validate early, fail gracefully.



Keep controllers thin, delegate to services.



Follow standards and use proven patterns.



Build for change, scale, and maintainability.

LAB OVERVIEW — REST ENDPOINTS AND REQUEST MAPPING

Ship `@RestController` Customer API with mapping annotations + MockMvc /

BUSINESS SCENARIO

- Northstar replaces any SOAP endpoint thinking with a clear Boot REST controller. Your lead freezes:

LEARNING OBJECTIVES

- Declare a `@RestController` for Customer collection and item resources Map HTTP verbs with `@GetMapping`, `@PostMapping`, and related annotations Bind `@PathVariable`, `@RequestParam`, and `@RequestBody` Return `ResponseEntity` with appropriate status codes and headers Verify endpoints with MockMvc and document CORS for Angular

WHAT YOU BUILD

- `examples/lab24-crm/` — controller, in-memory service, tests | Live GET/POST · MockMvc green · Angular CORS documented |

KEY TAKEAWAY This Module 24 lab implements the Customer HTTP surface with Spring MVC: `@RestController`, method mappings, binding annotations, `ResponseEntity`, MockMvc tests, and a short CORS note so the ****Angular**** SPA can call the API during local development.



LAB OVERVIEW – SPRING MVC REST ENDPOINT DEVELOPMENT



Module 24
REST Endpoints and
Request Mapping

In this lab, you will build and test REST endpoints using Spring MVC. You will implement controllers, handle requests and responses, validate inputs, and test your APIs with Postman/cURL.

BUSINESS SCENARIO

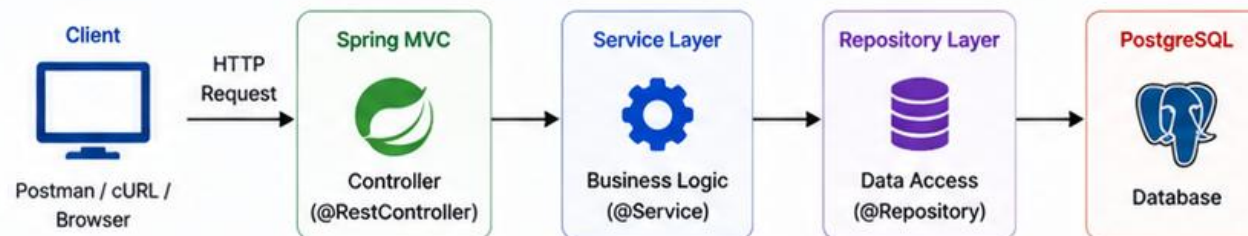


You are building a Company **Employee Management API** that allows clients to manage employees, departments, and their details.

LAB OBJECTIVES

- ✓ Create REST Controllers using Spring MVC annotations
- ✓ Implement CRUD endpoints for employees
- ✓ Handle path variables, query parameters, and request bodies
- ✓ Return appropriate HTTP status codes and response bodies
- ✓ Validate input and handle errors gracefully
- ✓ Test your endpoints using Postman/cURL

LAB ARCHITECTURE



EMPLOYEE API ENDPOINTS (EXAMPLES)

Method	Endpoint	Description	Request	Response	Status Codes
GET	/api/employees	Get all employees	–	List<Employee>	200 OK
GET	/api/employees/{id}	Get employee by ID	Path Variable	Employee	200 OK, 404 Not Found
POST	/api/employees	Create a new employee	Employee (JSON)	Employee	201 Created, 400 Bad Request
PUT	/api/employees/{id}	Replace an employee	Employee (JSON)	Employee	200 OK, 404 Not Found
PATCH	/api/employees/{id}	Update partial fields	Employee (JSON)	Employee	200 OK, 404 Not Found
DELETE	/api/employees/{id}	Delete an employee	–	–	204 No Content, 404 Not Found

TECHNOLOGIES USED

- Spring Boot
- Spring MVC
- PostgreSQL
- Jackson (JSON)
- Validation (Jakarta Validation)
- Postman / cURL

LAB DELIVERABLES

- ✓ Spring Boot project with REST controllers
- ✓ CRUD endpoints for Employee resource
- ✓ Input validation and error handling
- ✓ Postman collection with test requests
- ✓ Sample cURL commands
- ✓ README with setup and test instructions



BEST PRACTICES



Follow REST naming conventions and use resource-based URIs.



Return meaningful HTTP status codes and consistent responses.



Validate inputs at the controller boundary and handle errors gracefully.



Keep controllers thin; delegate business logic to services.



Test early and often using Postman/cURL and MockMvc.



TIP: Use layered architecture and clear separation of concerns to build maintainable and testable REST APIs.

LAB TASKS AND DELIVERABLES

REST Endpoints and Request Mapping — implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Open starter/README.md.
2	Copy into java-bootcamp/examples/lab24-crm/.
3	Fill controller // TODO mappings + MockMvc assertions.
4	Run mvn -B test and a live GET; evidence under notes/screenshots/lab-24/.
5	Mark timed-path Pass criteria.
6	CustomerController with @RestController + mapping annotations
7	PathVariable get, RequestBody create, optional RequestParam list filter
8	ResponseEntity with correct status codes (200/201/404)
9	CustomerControllerMockMvcTest green
10	Live curl/Invoke-WebRequest evidence for CUS-1001 / CUS-1002
11	CORS note for Angular (docs/cors-angular.md)
12	No secrets or target/ committed

EXPECTED DELIVERABLES

- CustomerController with @RestController + mapping annotations PathVariable get, RequestBody create, optional RequestParam list filter ResponseEntity with correct status codes (200/201/404) CustomerControllerMockMvcTest green Live curl/Invoke-WebRequest evidence for CUS-1001 / CUS-1002 CORS note for Angular (docs/cors-angular.md) No secrets or target/ committed

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs/Evidence 10

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor Lab 24; complete pre-lab exercises 1→6 first, then the graded lab.

Lab Tasks and Deliverables

In this lab, you will design a REST API for an Employee Management system following REST architecture principles and best practices.



Lab Outcome

A well-designed [REST API specification](#) documented using OpenAPI and aligned with enterprise best practices.



LAB TASKS

- 1 Review the business scenario and identify core resources.
- 2 Design resource URIs following REST naming standards.
- 3 Map HTTP methods to CRUD operations for each resource.
- 4 Define request and response structures using JSON.
- 5 Specify appropriate HTTP status codes for all operations.
- 6 Design error response format for client and server errors.
- 7 Add pagination, filtering and sorting to list endpoints.
- 8 Document the API using OpenAPI (Swagger).
- 9 Validate the API design using an OpenAPI validator.



DELIVERABLES

- ✓ **Resource Inventory:** List of identified resources and their purpose.
- ✓ **API Endpoint Catalog:** URI table with HTTP methods and descriptions.
- ✓ **Request & Response Models:** JSON schemas for key operations.
- ✓ **Error Response Design:** Standardized error format and status code mapping.
- ✓ **OpenAPI Specification:** Complete OpenAPI (Swagger) document.
- ✓ **Validation Report:** Successful validation output from OpenAPI validator.
- ✓ **Design Review Notes:** Key REST design decisions and rationale.



LAB TIPS



Start with resources, not database tables.



Keep URIs clean, consistent and noun-based.



Use HTTP methods as designed.



Document everything using OpenAPI.



Follow best practices used in real-world APIs.

MODULE 24 SUMMARY

Spring MVC internals, mapping annotations, data binding, responses, and controller testing, tied together.

KEY CONCEPTS COVERED



Spring MVC Pipeline

DispatcherServlet, handler mapping, controller, and message converter, in that order.



Mapping Annotations

@RestController plus @GetMapping/@PostMapping/etc., mapped straight from Module 13's HTTP methods.



Data Binding

@PathVariable, @RequestParam, and @RequestBody, each with a distinct, deliberate purpose.



Response Shaping

ResponseEntity gives full control over status codes and headers, never left to a default.



Validation and Errors

@Valid plus a centralized @ControllerAdvice keep every failure response consistent.



Controller Testing

MockMvc verifies HTTP behavior fast, without a running server.

MODULE FLOW RECAP

1

1. Pipeline

2

2. Mapping

3

3. Binding

4

4. Responses

5

5. Testing

KEY TAKEAWAY A REST controller layer is only as good as its discipline: thin methods, correct status codes, and tests that prove both.



MODULE SUMMARY

In this module, you learned how to design RESTful APIs that are resource-oriented, consistent, and aligned with enterprise best practices.



KEY TAKEAWAYS



REST Principles and Constraints

You explored the REST architectural style, its constraints, and how they ensure scalable and maintainable APIs.



Resource-Based Design

You learned how to identify resources and design clean, consistent, and hierarchical URIs.



HTTP Methods and Semantics

You applied the correct HTTP methods for CRUD operations and understood safe and idempotent operations.



Requests and Responses

You defined request/response structures, status codes, headers, and standardized error response formats.



API Documentation and Versioning

You created OpenAPI specifications and applied versioning strategies to support API evolution.



WHAT YOU CAN DO NOW



Design **resource-oriented** REST APIs with clear and meaningful URIs.



Use the correct **HTTP methods** and **status codes** for all operations.



Design **consistent** request and response payloads using **JSON**.



Implement **pagination**, **filtering**, and **sorting** for collection endpoints.



Apply **API versioning** strategies to support backward compatibility.



Document APIs using **OpenAPI** and validate using tools.



WHY IT MATTERS

Well-designed REST APIs improve interoperability, scalability, and developer experience. They provide a stable contract between systems and enable long-term maintainability.



KNOWLEDGE CHECK (1 OF 2)

Test your understanding of Spring MVC's pipeline, mapping annotations, and data binding.

1. What is `@RestController` equivalent to?

- A. `@Controller` alone
- B. `@Controller` plus `@ResponseBody` on every method**
- C. `@Service` plus `@Repository`
- D. `@Component` with no special behavior

3. Which annotation binds the 42 in `GET /employees/42` to a method parameter?

- A. `@RequestParam`
- B. `@RequestBody`
- C. `@PathVariable`**
- D. `@ModelAttribute`

2. Two methods share the path `"/employees/{id}"`, one `@GetMapping` and one `@DeleteMapping`. How does Spring choose which one runs?

- A. It always runs the first one declared
- B. It matches the request's HTTP method to one mapping**
- C. It runs both and merges the results
- D. It throws an error at startup

4. A client sends a PUT to a path where only GET and DELETE are mapped. What status code should result?

- A. 404 Not Found
- B. 405 Method Not Allowed**
- C. 400 Bad Request
- D. 200 OK

KEY TAKEAWAY `@RestController` bakes in `@ResponseBody`, method matching narrows a shared path, `@PathVariable` binds identity, and a valid path with the wrong method is 405, not 404.

KNOWLEDGE CHECK (2 OF 2)

Test your understanding of response shaping, validation, exception handling, and testing.

1. Why should a controller never return a JPA entity directly?

- A. It is slower than returning a DTO
- B. Lazy-loading errors and leaked internal fields**
- C. Jackson cannot serialize entities at all
- D. Entities are always immutable

3. What is the main advantage of @ControllerAdvice for exception handling?

- A. It makes controllers run faster
- B. It centralizes error handling across all controllers**
- C. It replaces the need for status codes
- D. It disables validation

2. What does adding @Valid before @RequestBody actually trigger?

- A. JSON syntax checking only
- B. Bean Validation against the DTO's constraint annotations**
- C. Automatic database persistence
- D. CORS header validation

4. Why does @WebMvcTest with MockMvc start faster than a full @SpringBootTest?

- A. It skips writing assertions
- B. It loads only the web layer, not the whole app**
- C. It runs tests in a separate process
- D. It disables Jackson serialization

KEY TAKEAWAY Never return entities directly, @Valid triggers Bean Validation (not just JSON parsing), @ControllerAdvice centralizes error handling, and MockMvc slice tests skip loading the whole app for speed.



KNOWLEDGE CHECK

? Test your understanding of REST API design concepts.
Choose the best answer for each question.

1 Which REST constraint ensures that clients can cache responses to improve performance?



- (A) Stateless
- (B) Cacheability
- (C) Layered System
- (D) Uniform Interface

2 Which HTTP method should be used to create a new resource?



- (A) GET
- (B) POST
- (C) PUT
- (D) DELETE

3 Which status code indicates that a request was successful and a resource was created?



- (A) 200 OK
- (B) 201 Created
- (C) 204 No Content
- (D) 400 Bad Request

4 What is the primary purpose of API versioning?



- (A) Improve server performance
- (B) Support backward compatibility
- (C) Reduce response size
- (D) Increase security

5 Which header is used by the client to specify the expected response format?



- (A) Content-Type
- (B) Accept
- (C) Authorization
- (D) Cache-Control

6 Which of the following is an example of a well-designed resource URI?



- (A) /getEmployeeData?id=123
- (B) /v1/employees/123
- (C) /employeeDetailById.php?id=123
- (D) /api/data/fetchEmployee/123



TIP

Good REST API design leads to scalable, maintainable, and easy-to-use systems. Always think resources, HTTP semantics, and consistency.



ANSWER KEY

1. B 2. B 3. B 4. B 5. B 6. B



MODULE 24 COMPLETE

REST Endpoints and Request Mapping

You can now build, bind, respond from, and test Spring MVC REST controllers end to end.